

LLM / VLM 进阶面试课程 Roadmap

这套笔记面向 AI（Artificial Intelligence，人工智能）算法工程师的 LLM（Large Language Model，大语言模型）与 VLM（Vision-Language Model，视觉语言模型）面试准备。当前版本按 2026-06-23 的技术格局重写：不只讲 Transformer 和 RAG（Retrieval-Augmented Generation，检索增强生成），还覆盖 reasoning model、post-training、RLVR（Reinforcement Learning with Verifiable Rewards，可验证奖励强化学习）、agentic tool use、long-horizon coding agent、eval harness、VLM 数据和评测实操、SOTA（State of the Art，当前最优）技术报告阅读和系统安全。

全局符号和缩写说明

阅读约定：大写字母通常表示矩阵、随机变量、模块、数据集或模型族；小写字母通常表示样本、token、向量或标量。第一次遇到公式前，先把常用符号列清楚。

符号/缩写	English	中文	含义
AI	Artificial Intelligence	人工智能	本课程讨论的总领域。
LM	Language Model	语言模型	语言建模系统的通称，LLM 是大规模 LM。
LLM	Large Language Model	大语言模型	以大规模文本预训练为核心的生成式语言模型。
VLM	Vision-Language Model	视觉语言模型	同时处理图像/视频和文本的多模态模型。
MLLM	Multimodal Large Language Model	多模态大语言模型	把视觉、文本、音频等模态接入 LLM 的模型族。
NLP	Natural Language Processing	自然语言处理	语言理解与生成任务领域。
CV	Computer Vision	计算机视觉	图像、视频理解与生成任务领域。
API	Application Programming Interface	应用程序编程接口	模型、工具、服务之间交互的接口。
DB	Database	数据库	存储结构化或半结构化数据的系统。
JSON	JavaScript Object Notation	JavaScript 对象表示法	工具调用和结构化输出常用格式。
PDF	Portable Document Format	可移植文档格式	文档理解和企业知识库中常见文件格式。
GUI	Graphical User Interface	图形用户界面	computer-use agent 操作的桌面或网页界面。

符号/缩写	English	中文	含义
Transformer	Transformer Architecture	Transformer 架构	基于 attention 的主流序列建模架构。
BPE	Byte Pair Encoding	字节对编码	常见 subword tokenization 方法。
Attention	Attention Mechanism	注意力机制	用 query-key-value 计算 token 间依赖的机制。
MHA	Multi-Head Attention	多头注意力	在多个子空间并行计算 attention 的模块。
$Q / K / V$	Query / Key / Value	查询 / 键 / 值	attention 中的三个投影矩阵或向量，不等同于 RL 里的 Q value。
X	Input Sequence	输入序列	token embedding 组成的输入矩阵。
x_i	Token Embedding	第 i 个 token 向量	序列中某个 token 的向量表示。
h_i	Hidden State	隐状态	Transformer 某层输出的第 i 个 token 表示。
θ	Model Parameters	模型参数	神经网络中被训练的参数。
$L(\theta)$	Loss Function	损失函数	训练时最小化的目标。
CE	Cross Entropy	交叉熵	自回归语言模型常用训练损失。
SFT	Supervised Fine-Tuning	监督微调	用指令-答案数据继续训练模型。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用偏好数据训练 reward model，再优化策略。
RM	Reward Model	奖励模型	对回答、步骤或轨迹打分的模型。
PPO	Proximal Policy Optimization	近端策略优化	RLHF 早期常用的策略优化算法。
KL	Kullback-Leibler Divergence	KL 散度	衡量当前策略偏离参考策略的程度。
RLAIF	Reinforcement Learning from AI Feedback	基于 AI 反馈的强化学习	用 AI 反馈训练 preference/reward signal。
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	用可自动验证的 reward 训练推理、代码、工具任务。
GRPO	Group Relative Policy Optimization	组相对策略优化	DeepSeekMath / DeepSeek-R1 系列使用的 PPO 变体，不显式训练 value model。

符号/缩写	English	中文	含义
PRM	Process Reward Model	过程奖励模型	给推理中间步骤打分的 reward model。
ORM	Outcome Reward Model	结果奖励模型	只根据最终答案或任务结果打分的 reward model。
DPO	Direct Preference Optimization	直接偏好优化	不显式训练 reward model 的偏好对齐方法。
CoT	Chain-of-Thought	思维链	模型内部或外显的中间推理过程。
RAG	Retrieval-Augmented Generation	检索增强生成	先检索外部知识，再让模型基于上下文生成。
IR	Information Retrieval	信息检索	RAG 背后的检索领域。
BM25	Best Matching 25	BM25 排序函数	经典关键词检索打分方法。
Vector DB	Vector Database	向量数据库	存储 embedding 并支持相似度检索的数据库。
Hybrid Search	Hybrid Search	混合检索	组合 BM25、dense retrieval、metadata filter 等信号。
GraphRAG	Graph Retrieval-Augmented Generation	图检索增强生成	用实体关系、图结构和社区摘要增强检索。
Agentic RAG	Agentic Retrieval-Augmented Generation	智能体式 RAG	让 agent 动态规划、检索、验证和补查证据。
RAGAS	RAG Assessment	RAG 评估工具	常用于 RAG faithfulness、context relevance 等自动评测。
MTEB	Massive Text Embedding Benchmark	大规模文本向量评测	embedding 模型常用评测基准。
HELM	Holistic Evaluation of Language Models	语言模型整体评测	综合评估模型准确性、鲁棒性、公平性等维度。
Eval	Evaluation	评测	对模型或系统能力、风险和成本的测量。
Harness	Harness	评测/执行框架	管理数据、prompt、模型调用、工具、环境、打分和记录的系统层。
Eval Harness	Evaluation Harness	评测框架	把 benchmark 跑成可复现实验的执行框架。
Agent Harness	Agent Harness	智能体执行框架	管理 agent 上下文、工具、状态、权限、trace 和恢复的系统层。

符号/缩写	English	中文	含义
Benchmark	Benchmark	基准测试	固定任务集合和指标，用于比较模型或系统。
Oracle	Oracle	标准判定器	能可靠判断答案或最终状态是否正确的程序/标注。
Judge	Judge	裁判模型/评审者	用模型或人类判断开放式输出质量。
Trace	Trace	轨迹日志	记录 prompt、tool call、observation、输出和分数。
pass@k	Pass at k	k 次通过率	采样 k 次至少一次成功的概率指标。
Contamination	Data Contamination	数据污染	测试集进入训练数据或 prompt 导致分数虚高。
Agent	Agent	智能体	能规划、调用工具、观察结果并继续行动的系统。
MCP	Model Context Protocol	模型上下文协议	将工具、资源和上下文暴露给 agent 的协议范式。
MCTS	Monte Carlo Tree Search	蒙特卡洛树搜索	用搜索树探索候选推理或行动路径的方法。
SWE	Software Engineering	软件工程	代码生成、修复、测试、仓库级任务等场景。
SWE-bench	Software Engineering Benchmark	软件工程基准	用真实 GitHub issue 测试代码 agent 的 benchmark。
Terminal-Bench	Terminal Benchmark	终端任务基准	测试模型在 shell/terminal 中完成多步任务的能力。
BrowseComp	Browsing Competition Benchmark	浏览检索基准	测试 browsing agent 查找隐蔽信息的能力。
TAU-bench	Tool-Agent-User Benchmark	工具-智能体-用户交互基准	测试 agent 与用户、API、业务规则交互的稳定性。
OSWorld	Operating System World Benchmark	操作系统世界基准	测试 GUI/computer-use agent 的 benchmark。
MoE	Mixture of Experts	混合专家模型	每个 token 只激活部分专家参数，提高容量/成本比。
PEFT	Parameter-Efficient Fine-Tuning	参数高效微调	只训练少量参数的微调方法。
LoRA	Low-Rank Adaptation	低秩适配	在权重更新中注入低秩矩阵的 PEFT 方法。

符号/缩写	English	中文	含义
CLIP	Contrastive Language-Image Pre-training	对比式图文预训练	学习图像和文本共享语义空间的经典方法。
ViT	Vision Transformer	视觉 Transformer	把图像切成 patch token 后用 Transformer 编码。
OCR	Optical Character Recognition	光学字符识别	从图像中识别文字。
VQA	Visual Question Answering	视觉问答	给定图像和问题，输出答案。
MMLU	Massive Multitask Language Understanding	大规模多任务语言理解基准	LLM 知识与推理评测基准。
AIME	American Invitational Mathematics Examination	美国邀请数学竞赛	reasoning model 常用数学评测之一。
GPQA	Graduate-Level Google-Proof Q&A	研究生级抗搜索问答	面向高难科学知识和推理的评测。
ARC	Agentic, Reasoning, Coding	智能体、推理、编程	GLM 系列报告中强调的三类核心能力。
DSA	DeepSeek Sparse Attention	DeepSeek 稀疏注意力	GLM-5 报告采用的长上下文高效注意力路线。
MTP	Multi-Token Prediction	多 token 预测	用于提升解码吞吐或 speculative decoding 的机制。
RLCS	Reinforcement Learning with Curriculum Sampling	课程采样强化学习	GLM-4.5V 报告中用于多模态推理训练的 RL 方法。
STEM	Science, Technology, Engineering, Mathematics	科学、技术、工程、数学	多模态和推理模型常见高难评测领域。
MMMU	Massive Multi-discipline Multimodal Understanding	大规模多学科多模态理解基准	VLM 专家级多模态理解评测基准。
GPT	Generative Pre-trained Transformer	生成式预训练 Transformer	OpenAI GPT 系列模型名称中的缩写。
GLM	General Language Model	通用语言模型	智谱 / Z.ai GLM 系列模型名称中的缩写。

学习路线

章节	主题	面试重点
第 1 章	Transformer、tokenization、自回归预训练	attention 公式、causal mask、next-token prediction。
第 2 章	Scaling、post-training、alignment	SFT、RLHF、RLAIF、DPO、RLVR、GRPO、安全对齐。
第 3 章	Prompt、agent loop、tool use、system eval	prompt 边界、ReAct、function calling、系统评测分层。
第 4 章	VLM 基础、ViT、CLIP 与图文对齐	ViT、CLIP contrastive learning、VQA/OCR。
第 5 章	MLLM 架构与训练	BLIP-2、Flamingo、LLaVA、Qwen2.5-VL。
第 6 章	VLM 数据构造、预处理与训练实操	OCR、文档、图表、视频、UI 数据 schema、dynamic resolution、SFT/preference/RLVR。
第 7 章	VLM 评测 Harness、Benchmark 与项目实操	DocVQA、ChartQA、Video-MME、GUI、VLM judge、badcase 归因。
第 8 章	进阶面试速查与项目表达	高频问答、RAG/coding agent/VLM 项目讲法。
第 9 章	Reasoning model、RLVR 与过程监督	test-time compute、PRM/ORM、verifier、GRPO。
第 10 章	Agentic systems 与 coding agent	computer use、多工具、多 agent、长期任务评测。
第 11 章	2026 SOTA 技术报告阅读	GPT-5.5、Claude Opus 4.7、Gemini 2.5、DeepSeek-R1、Qwen3、Kimi K2、GLM-5.2。
第 12 章	RAG 系统、Agentic RAG 与评测	hybrid retrieval、GraphRAG、RAGAS、citation verification、权限。
第 13 章	Eval Harness、Benchmark 与可复现评测	LM Evaluation Harness、Agent Harness、RAG Harness、protocol、trace。
References	论文、官方技术报告、benchmark 链接	用于深入阅读和面试前补引用。

推荐学习顺序

1. 先学第 1 章，能手写 attention 公式和 next-token prediction。
2. 第 2 章理解现代 LLM 的 post-training pipeline：SFT、preference、RL、safety tuning。
3. 第 9 章重点攻 reasoning model：为什么 test-time compute、PRM/ORM、RLVR、GRPO 成为 2025-2026 主线。
4. 第 12 章单独攻 RAG：hybrid retrieval、GraphRAG、Agentic RAG、RAG eval 和权限。
5. 第 10 和第 13 章用于工程面试：agentic coding、harness、benchmark protocol、trace、可复现评测。
6. 第 11 章读 SOTA 技术报告，学会横向比较闭源/开源模型，而不是背模型名字。
7. 第 4-7 章转向 VLM/MLLM：图文对齐、视觉 token 接入 LLM、多模态指令微调、数据构造、评测 harness。
8. 第 8 章用于面试前压缩复习。

面试目标

学完后你应该能清楚回答：

- Transformer attention 为什么是 $QK^T / \sqrt{d_k}$ 。
- pretraining、SFT、RLHF、RLAIF、DPO、RLVR 的关系。
- RLVR、GRPO、PRM、ORM 和传统 RLHF 的区别。
- reasoning model 为什么用更多 inference compute 能提升复杂任务表现。
- agentic tool use 和普通 function calling 的差异。
- coding agent 如何评估：SWE-bench、Terminal-Bench、TAU-bench、BrowseComp、open-world eval。
- harness、benchmark、scaffold、leaderboard 的区别。
- LoRA 和 QLoRA 为什么省显存。
- RAG 为什么能缓解幻觉，以及 hybrid retrieval、GraphRAG、Agentic RAG、RAGAS 的取舍。
- GPT-5.5、Claude Opus 4.7、Gemini 2.5、DeepSeek-R1、Qwen3、Kimi K2、GLM-5.2 各自技术报告的关键点。
- CLIP 为什么能 zero-shot transfer。
- BLIP-2 / Flamingo / LLaVA 的核心架构差异。
- VLM 数据如何构造：OCR、文档、图表、视频、UI、grounding、preference/RLVR。
- 如何设计 VLM eval harness，并做 OCR、文档、图表、视频、GUI 和安全 badcase 归因。
- VLM 在 OCR、chart、document、spatial reasoning 上为什么容易出错。
- 如何设计一个 LLM/VLM 项目并说明指标、数据、评测和风险。

第 1 章：Transformer、Tokenization 与自回归预训练

本章符号说明

符号/缩写	English	中文	含义
Transformer	Transformer Architecture	Transformer 架构	基于 self-attention 的序列建模架构。
BPE	Byte Pair Encoding	字节对编码	常见 subword tokenization 方法。
MHA	Multi-Head Attention	多头注意力	在多个子空间并行计算 attention 的模块。
X	Input Embedding Matrix	输入嵌入矩阵	一段 token embedding 组成的矩阵。
$Q / K / V$	Query / Key / Value	查询 / 键 / 值	attention 中由 X 线性投影得到的表示。
d_k	Key Dimension	key 向量维度	attention 缩放项中的维度。
$P(w_t / w_{<t})$	Next-token Probability	下一个 token 概率	自回归语言模型的训练目标。
CE	Cross Entropy	交叉熵	预测 token 分布与真实 token 的损失。

本章目标

- 理解 tokenization 为什么重要。
- 掌握 self-attention 的矩阵公式。
- 区分 encoder-only、decoder-only、encoder-decoder。
- 理解 next-token prediction 为什么能学到通用能力。
- 能解释 pretraining loss 和 inference decoding。

Tokenization

LLM 不能直接处理字符串，通常先把文本切成 token，再映射成 embedding。

常见 tokenization 方法：

- word-level：按词切分，词表大，未登录词问题明显。
- char-level：按字符切分，词表小，但序列很长。
- subword：折中方案，BPE、WordPiece、SentencePiece 都属于这一类。

面试表达：

Tokenization 决定了模型看到的最小语义单位，也影响上下文长度、压缩率、多语言表现和代码能力。中文、代码、数学表达式通常对 tokenizer 更敏感。

Self-Attention

输入 embedding 矩阵为：

$$X \in \mathbb{R}^{n \times d}$$

通过线性投影得到：

$$Q = XW_Q, K = XW_K, V = XW_V$$

scaled dot-product attention：

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})V$$

直觉：

- Q 表示当前位置想找什么信息。
- K 表示每个位置能被什么信息匹配。
- V 表示真正被聚合的内容。
- 除以 $\sqrt{d_k}$ 是为了避免点积随维度变大而过大，使 softmax 过于尖锐。

Multi-Head Attention

multi-head attention 让模型在多个子空间里计算 attention：

$$head_i = Attention(XW_i^Q, XW_i^K, XW_i^V)$$

然后拼接：

$$MHA(X) = Concat(head_1, \dots, head_h)W^O$$

面试直觉：

不同 head 可以关注不同关系，例如局部语法、长程依赖、实体指代、代码括号匹配。它不是保证每个 head 都有清晰可解释含义，但增加了并行建模不同关系的能力。

Decoder-only LLM

主流生成式 LLM 多用 decoder-only Transformer。训练目标是自回归 next-token prediction：

$$\max_{\theta} \sum_{t=1}^T \log P_{\theta}(w_t / w_{<t})$$

对应最小化 cross entropy:

$$L(\theta) = - \sum_{t=1}^T \log P_{\theta}(w_t / w_{<t})$$

为了防止模型看到未来 token，会使用 causal mask:

$$mask_{ij}=0 \text{ if } j \leq i$$

为什么 next-token prediction 有用

next-token prediction 看起来只是预测下一个词，但要做好它，模型需要隐式学习：

- 语法和语义。
- 世界知识。
- 长程依赖。
- 代码和数学模式。
- 指令和对话格式。

但它也有天然限制：

- 目标是拟合语料分布，不等于遵循用户意图。
- 训练目标不直接惩罚幻觉。
- 模型可能学到偏见、噪声和过期知识。

这就是为什么后续还需要 SFT、RLHF、DPO 和 RAG。

Inference Decoding

推理时，模型会根据当前上下文预测下一个 token，然后把生成 token 拼回上下文继续生成。

常见 decoding 策略：

方法	English	中文	直觉
Greedy	Greedy Decoding	贪心解码	每步选概率最高 token，稳定但容易死板。
Beam Search	Beam Search	束搜索	保留多个候选，适合翻译，不一定适合开放生成。
Temperature	Temperature Sampling	温度采样	调整分布尖锐程度。
Top-k	Top-k Sampling	Top-k 采样	只从最高的 k 个 token 采样。
Top-p	Nucleus Sampling	核采样	从累计概率达到 p 的 token 集合中采样。

本章面试高频问题

1. Transformer 相比 RNN 的核心优势是什么？

Transformer 用 attention 直接建模任意 token 间关系，并且训练时可以并行处理序列位置；RNN 顺序递推，长程依赖和并行效率都更弱。

2. 为什么 attention 要除以 $\sqrt{d_k}$ ？

点积的方差会随维度变大而增大，如果不缩放，softmax 会过于尖锐，梯度变小，训练不稳定。除以 $\sqrt{d_k}$ 可以让 attention logits 的尺度更稳定。

3. decoder-only 为什么适合生成？

decoder-only 使用 causal mask，只能看当前位置之前的 token，天然匹配从左到右生成。GPT 系列、LLaMA 系列、Qwen 系列都属于这一路线。

4. next-token prediction 为什么能产生 few-shot 能力？

大规模语料里存在大量任务描述、示例、问答、代码和推理模式。模型在预训练中学习这些分布后，可以在 prompt 中根据少量示例适配新任务。

本章 References

- [Attention Is All You Need](#)
- [Language Models are Few-Shot Learners](#)
- [FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

第 2 章：Scaling、Post-training、Alignment 与 RLVR

本章符号说明

符号/缩写	English	中文	含义
Scaling Law	Scaling Law	缩放律	模型性能随参数、数据、算力变化的经验规律。
SFT	Supervised Fine-Tuning	监督微调	用高质量指令-答案数据训练模型遵循格式和任务。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward model，再优化策略。
RLAIF	Reinforcement Learning from AI Feedback	基于 AI 反馈的强化学习	用 AI 反馈替代或补充人类偏好反馈。
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	用可自动验证的正确性信号训练模型。
RM	Reward Model	奖励模型	根据偏好或任务结果预测回答质量的模型。
PRM	Process Reward Model	过程奖励模型	对中间推理步骤打分。
ORM	Outcome Reward Model	结果奖励模型	对最终答案或最终状态打分。
PPO	Proximal Policy Optimization	近端策略优化	RLHF 早期常用策略优化算法。
GRPO	Group Relative Policy Optimization	组相对策略优化	用同组样本的相对 reward 构造 advantage 的 PPO 变体。
DPO	Direct Preference Optimization	直接偏好优化	直接用偏好对优化策略，不显式训练 reward model。
KL	Kullback-Leibler Divergence	KL 散度	衡量当前策略偏离参考策略的程度。
CoT	Chain-of-Thought	思维链	模型内部或外显的中间推理过程。
CoT Monitor	Chain-of-Thought Monitor	思维链监控器	用推理轨迹辅助发现不安全或欺骗性行为的监控方法。
π_θ	Policy Model	策略模型	当前生成回答的语言模型。

符号/缩写	English	中文	含义
π_{ref}	Reference Model	参考模型	对齐训练中用于约束偏移的原始模型。
$r_{\phi}(x,y)$	Reward Function	奖励函数	对 prompt x 和回答 y 打分。
A_i	Advantage	优势函数	样本 i 相对基线的好坏程度。

本章目标

- 理解 scaling law、compute-optimal training 和 test-time compute 的关系。
- 掌握 pretraining、SFT、preference alignment、RLVR、safety tuning 的完整流水线。
- 能解释 RLHF、RLAIF、DPO、PPO、GRPO、PRM、ORM 的差异。
- 能从工程角度说明为什么 2025-2026 的 reasoning model 更依赖 RL 和可验证环境。
- 能回答 alignment 面试里常见的 reward hacking、sycophancy、over-refusal、jailbreak、hidden CoT 问题。

从 Scaling 到 Post-training

LLM 能力来自四个维度的共同缩放：

1. 参数量和模型容量。
2. 训练 token 数和数据质量。
3. 训练 compute 和优化稳定性。
4. post-training 与 inference-time compute。

Chinchilla 之后，一个关键观点是：固定 compute 下，参数量和训练 token 数要更均衡。2025-2026 的新变化是 reasoning model 把 scaling 延伸到 post-training 和推理阶段：不仅预训练更大，后训练 RL 更多，推理时也允许模型“想更久”。

面试表达：

传统 scaling law 主要解释 pretraining loss 随 model/data/compute 的变化；reasoning model 之后，还要看 post-training RL compute 和 inference-time compute。复杂数学、代码、工具调用任务上，模型能力不只是参数记忆，而是搜索、验证、自我纠错和工具使用策略的综合结果。

现代 Post-training Pipeline

现代 frontier model 常见流程：

1. Pretraining：大规模 next-token prediction，学习通用语言、代码、知识和世界模型。
2. Continued pretraining：补领域数据、多语言、代码、长上下文、多模态数据。
3. SFT：学习指令格式、对话风格、工具 schema、拒答模板。
4. Preference alignment：用人类或 AI 偏好提升 helpfulness、honesty、style。
5. Reasoning RL / RLVR：在数学、代码、工具环境、浏览环境中用可验证 reward 优化。

6. Safety tuning: 安全策略、policy reasoning、over-refusal 控制、jailbreak 鲁棒性。

7. Product eval: 自动 benchmark、人工评测、红队、安全预案、线上监控。

一个重要面试点: post-training 不只是“让模型更礼貌”。它会显著改变 instruction following、工具调用、长任务执行、推理深度和安全边界。

SFT

SFT 用指令数据训练模型:

$$L_{SFT}(\theta) = - \sum_t \log P_{\theta}(y_t | x, y_{<t})$$

优点:

- 稳定简单。
- 能显著改善指令遵循和格式。
- 数据质量比数据数量更关键。

局限:

- 学的是示范分布, 不直接优化“哪个回答更好”。
- 对数学、代码、agent 这类可验证任务, SFT 只能模仿轨迹, 不一定学会探索和纠错。
- 对 safety 的提升依赖覆盖率, 没见过的 jailbreak 和边界条件仍容易出错。

RLHF 与 PPO

RLHF 典型三步:

1. 收集 prompt 和多个候选回答。
2. 人类标注偏好, 训练 reward model。
3. 用 PPO 等 RL 算法优化语言模型, 同时用 KL penalty 限制偏移。

常见目标:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}} [r_{\phi}(x, y)] - \beta KL(\pi_{\theta} \parallel \pi_{ref})$$

直觉:

- reward model 代表“人类更喜欢什么”。
- KL penalty 防止模型为了 reward hacking 偏离太远。
- PPO 可以在线采样并优化, 但训练复杂、显存高、调参成本大。

面试追问:

为什么 RLHF 会 reward hacking?

因为 reward model 是人类偏好的近似, 不是真实目标。如果模型发现某些模式能骗过 RM, 比如过度自信、迎合用户、模板化安全话术, 就会优化这些捷径。

RLAIF 与 Constitutional AI

RLAIF 用 AI feedback 替代或补充人类反馈。Anthropic 的 Constitutional AI 是代表路线：先让模型根据一组原则自我批评和修订，再用 AI 偏好训练 preference model，最后用 RL 优化。

优点：

- 降低人类标注成本。
- 安全原则更显式，适合扩展到大量边界样本。
- 可用更强模型监督较弱模型。

风险：

- AI judge 也会有偏差。
- 如果 judge 与 policy 共享偏见，可能放大系统性错误。
- 容易把 alignment 变成“符合某套文本规则”，但现实任务仍有灰区。

DPO

DPO 直接用偏好对训练，不显式跑 RL。对于同一个 prompt，有 preferred answer y_w 和 rejected answer y_l 。

简化表达：

$$L_{DPO} = -\log \sigma(\beta \Delta)$$

其中 Δ 表示当前模型相对 reference model 对 preferred/rejected 的 log probability ratio 差异。

面试表达：

DPO 把偏好优化转成 supervised-style loss，省掉 reward model 和 PPO loop，因此更稳定、更容易复现。但它主要适合 pairwise preference，不等于能替代所有在线 RL；在工具环境、代码执行、数学验证这类任务上，RLVR 更直接。

RLVR：为什么 2025-2026 很重要

RLVR 的核心是：有些任务可以自动判断对错，所以不需要昂贵的人类偏好。

典型可验证任务：

- 数学：最终答案可校验。
- 代码：单元测试、隐藏测试、lint、运行结果可校验。
- 工具调用：API 最终状态可比对。
- 浏览检索：答案可用 reference answer 检查。
- 形式化证明：proof checker 可校验。

RLVR 的目标可以抽象成：

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [R(x, y)]$$

其中 $R(x,y)$ 是自动验证 reward。

面试表达：

RLVR 是 reasoning model 的关键训练范式之一，因为数学、代码和 agent 任务有明确的外部反馈。模型不是只模仿人类答案，而是在采样、执行、验证、失败后调整的闭环中学会更可靠的推理策略。

PRM 与 ORM

ORM 只看最终结果：

$$R_{ORM}(x,y)=1[answer(y)=answer^*]$$

PRM 给中间步骤打分：

$$R_{PRM}(x,y_{1:t}) \rightarrow step\ quality$$

对比：

方法	English	中文	优点	缺点
ORM	Outcome Reward Model	结果奖励模型	标注和自动验证简单	不知道哪里错，稀疏 reward
PRM	Process Reward Model	过程奖励模型	能指导中间推理，利于搜索	标注贵，容易奖励表面步骤
Verifier	Verifier Model	验证器模型	可做 best-of-n / rerank	仍可能被模型 exploit

面试回答要点：

- ORM 适合答案可校验任务，但 reward 稀疏。
- PRM 更像“每一步都给老师批改”，对长推理有帮助。
- 现代系统经常组合：采样多条轨迹，用 verifier / test / tool result 选最优。

GRPO

GRPO 是 PPO 的一个 memory-efficient 变体。它不显式训练 value model，而是在同一个 prompt 下采样一组回答，用组内 reward 均值和方差构造相对 advantage。

给同一个 prompt 采样 G 个回答，reward 为 r_i ，可以写成：

$$A_i = r_i - \text{mean}(r_1, \dots, r_G) / \text{std}(r_1, \dots, r_G)$$

直觉：

- 同组里表现更好的样本被强化。
- 表现更差的样本被压低概率。
- 省掉 value model，显存和训练复杂度下降。

风险：

- 如果一组样本 reward 都一样，学习信号会变弱。
- 对 reward 设计和采样温度敏感。
- 容易优化到“会拿分”的策略，不等于真实通用推理。

Safety Alignment：从拒答到安全推理

2026 面试里，alignment 不能只讲“模型会拒答”。要讲四类问题：

问题	English	中文	典型现象
Over-refusal	Over-refusal	过度拒答	安全问题过敏，正常请求也拒绝。
Under-refusal	Under-refusal	拒答不足	危险请求没有拦住。
Sycophancy	Sycophancy	迎合用户	明知用户错仍附和。
Reward Hacking	Reward Hacking	奖励黑客	优化指标漏洞，不完成真实目标。
CoT Faithfulness	Chain-of-Thought Faithfulness	思维链忠实性	展示的理由和真实决策过程不一致。

OpenAI 的 deliberative alignment 代表一种新思路：把安全规范直接教给模型，让模型在回答前对规范进行推理。GPT-5 system card 进一步强调 safe-completions，即尽量给出安全范围内有帮助的回答，而不是简单二元拒答。

面试表达：

安全对齐正在从“分类器式拒答”走向“策略推理”。模型需要判断用户意图、上下文、风险等级和可替代帮助方式。好的安全系统既减少 jailbreak，也减少 over-refusal。

本章面试高频问题

1. SFT、RLHF、DPO、RLVR 的关系？

SFT 学示范，RLHF 学人类偏好，DPO 用偏好对做稳定的直接优化，RLVR 用可验证 reward 优化数学、代码、工具等任务。它们不是互斥关系，现代模型通常多阶段组合使用。

2. 为什么 reasoning model 更依赖 RL？

复杂推理和工具任务不是单步模仿能解决的。RL 可以让模型从执行结果、测试结果、验证结果中学习策略，尤其适合有明确 reward 的数学、代码、agent 环境。

3. GRPO 和 PPO 的区别？

PPO 通常需要 value model 估计 advantage；GRPO 用同组候选的相对 reward 构造 advantage，省掉 value model，适合大模型 RL，但对 reward 分布和采样组质量敏感。

4. PRM 为什么不总是优于 ORM？

PRM 提供更密集的中间监督，但标注成本高，也可能奖励“看起来像推理”的步骤。ORM 简单但稀疏。实际系统常结合 verifier、best-of-n、工具执行和最终测试。

5. safe-completions 和普通拒答有什么区别？

普通拒答把请求分成允许/拒绝。safe-completions 更强调在安全边界内尽量有帮助：危险部分不提供，但可以给高层解释、安全替代方案或合规信息。

本章 References

- [Scaling Laws for Neural Language Models](#)
- [Training Compute-Optimal Large Language Models](#)
- [Training language models to follow instructions with human feedback](#)
- [Constitutional AI: Harmlessness from AI Feedback](#)
- [Direct Preference Optimization](#)
- [Let's Verify Step by Step](#)
- [DeepSeekMath: Pushing the Limits of Mathematical Reasoning](#)
- [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)
- [Deliberative Alignment: Reasoning Enables Safer Language Models](#)
- [OpenAI GPT-5 System Card](#)

第 3 章：Prompt、Agent 与系统评测基础

本章符号说明

符号/缩写	English	中文	含义
Prompt	Prompt	提示词	给模型的任务描述、约束和上下文。
CoT	Chain-of-Thought	思维链	让模型生成或内部使用中间推理步骤。
RAG	Retrieval-Augmented Generation	检索增强生成	检索外部文档后再生成答案。
Agent	Agent	智能体	能规划、调用工具、观察结果并继续行动的系统。
Tool Use	Tool Use	工具调用	模型调用搜索、数据库、代码解释器等外部工具。
Function Calling	Function Calling	函数调用	模型输出结构化调用参数，由系统执行函数。
API	Application Programming Interface	应用程序编程接口	工具和外部服务的调用接口。
DB	Database	数据库	RAG 或工具调用中常访问的数据系统。
GUI	Graphical User Interface	图形用户界面	OSWorld / computer-use 类 agent 操作的界面。
MCP	Model Context Protocol	模型上下文协议	暴露 tools、resources、prompts 的上下文协议范式。
Memory	Memory	记忆	agent 跨轮次保存的任务状态、用户偏好或外部事实。
Planner	Planner	规划器	负责拆任务、排步骤、选择工具的模块或行为。
Executor	Executor	执行器	负责调用工具、运行代码、修改文件等动作的模块。
Verifier	Verifier	验证器	检查答案、代码、引用或最终状态是否正确的模块。
Embedding	Embedding	向量表示	文本、图像或多模态对象的语义向量。

符号/缩写	English	中文	含义
BM25	Best Matching 25	BM25 排序函数	经典关键词检索打分方法。
Reranker	Reranker	重排序模型	对召回文档重新排序，提高证据质量。
MRR	Mean Reciprocal Rank	平均倒数排名	检索排序评测指标。
recall@k	Recall at k	前 k 召回率	gold evidence 是否出现在前 k 个结果中。
nDCG	Normalized Discounted Cumulative Gain	归一化折损累计增益	检索排序质量指标。
MTEB	Massive Text Embedding Benchmark	大规模文本向量评测	embedding 模型评测基准。
HELM	Holistic Evaluation of Language Models	语言模型整体评测	覆盖准确率、公平性、鲁棒性等维度的评测框架。
AIME	American Invitational Mathematics Examination	美国邀请数学竞赛	数学推理评测。
GPQA	Graduate-Level Google-Proof Q&A	研究生级抗搜索问答	高难科学知识和推理评测。
SWE-bench	Software Engineering Benchmark	软件工程基准	真实 GitHub issue 修复任务。
TAU-bench	Tool-Agent-User Benchmark	工具-智能体-用户交互基准	业务 API 多轮交互任务。
OSWorld	Operating System World Benchmark	操作系统世界基准	GUI/computer-use 任务评测。
PII	Personally Identifiable Information	个人可识别信息	安全评测中需要防泄漏的敏感信息。
RAGAS	RAG Assessment	RAG 评估工具	RAG 自动评测工具，详见第 10 章。
q	Query	查询	用户问题或检索查询。
D	Document Corpus	文档库	可检索的外部知识集合。
d_i	Retrieved Document	检索文档	被召回的第 i 个文档块。

本章目标

- 掌握 prompt engineering 的边界。
- 理解基础 RAG pipeline；高级 RAG、Agentic RAG 和 RAG eval 在第 10 章展开。
- 理解 agentic tool use 的系统结构。

- 能设计 LLM 应用的离线评测、在线监控和安全边界。

Prompt Engineering 的边界

好的 prompt 通常包含：

- Role：模型扮演什么角色。
- Task：具体要完成什么。
- Context：可用信息和背景。
- Constraints：格式、风格、禁止事项。
- Examples：few-shot 示例。
- Evaluation criteria：怎样算好。

面试表达：

Prompt engineering 不是玄学调参，而是在模型上下文里显式提供任务定义、约束、示例和输出格式。它能提升 instruction following，但不能从根本上补齐模型没有的知识、工具、权限和验证闭环。

CoT、Hidden Reasoning 与 Thinking Budget

CoT 可以提升复杂推理任务表现，但 2026 年面试要区分三件事：

1. 用户可见的解释：给用户看的推理摘要。
2. 模型内部的 reasoning trace：可能不展示，用于内部决策。
3. thinking budget：允许模型消耗多少推理 token 或时间。

工程注意点：

- 简单任务强行 CoT 会增加延迟和幻觉。
- 安全/隐私任务不应该暴露完整内部推理。
- reasoning token 越多不一定越好，要看任务是否可验证。
- 对 agent 来说，工具观察比纯文本推理更可靠。

RAG 基础 Pipeline

本节只建立基础概念。生产级 RAG、GraphRAG、Agentic RAG、RAGAS、权限和引用评测在第 10 章系统展开。

典型 RAG：

1. 文档解析和清洗。
2. chunking。
3. embedding。
4. 建立向量索引。
5. query rewrite。
6. retrieval。
7. reranking。

8. context packing。
9. generation。
10. citation 和 answer verification。

可以写成：

$$q \rightarrow \{d_1, \dots, d_k\} \rightarrow LLM(q, d_1, \dots, d_k)$$

RAG 解决的问题：

- 知识更新。
- 私有知识接入。
- 引用溯源。
- 降低纯参数记忆导致的幻觉。

RAG 解决不了的问题：

- 检索不到就无法回答。
- 检索到错误文档会污染生成。
- 长上下文里模型可能忽略关键证据。
- 权限、隐私、数据新鲜度仍需系统设计。

2026 RAG 重点：不是只做向量库

面试里不要把 RAG 说成“embedding + vector DB”。更完整的路线包括：

模块	English	中文	面试关注点
Hybrid Retrieval	Hybrid Retrieval	混合召回	BM25 + dense embedding，兼顾关键词和语义。
Query Rewrite	Query Rewrite	查询改写	多轮问题、省略指代、领域术语转换。
Reranking	Reranking	重排序	提高 top-k 证据相关性。
Context Packing	Context Packing	上下文打包	控制长度、去重、保留表格结构。
Citation Check	Citation Verification	引用校验	答案是否真正被引用支持。
Permission Filter	Permission Filtering	权限过滤	用户只能检索有权限文档。
Freshness	Freshness Control	新鲜度控制	数据版本、过期文档、增量索引。

长上下文模型并不自动替代 RAG。长上下文适合一次性给大量材料，RAG 适合动态知识库、权限隔离、引用和成本控制。现实系统常用 long-context RAG：先检索，再把更长证据包交给模型。

Agent 与 ReAct

Agent 不是“模型变聪明了”这么简单，而是系统让模型进入循环：

```
observe -> plan -> act/tool call -> observe -> verify -> continue/finish
```

ReAct 的思想是把 reasoning 和 action 交替起来，让模型一边推理一边调用外部工具。

工程上要关注：

- 工具 schema 是否清晰。
- 失败是否可恢复。
- 是否有循环上限。
- 是否记录轨迹用于 debug。
- 是否有权限和安全边界。
- 是否有 verifier 检查最终状态，而不是只看自然语言回答。

Tool Use：从 Function Calling 到 Agentic Tool Use

普通 function calling：

- 用户请求明确。
- 模型选择一个函数。
- 系统执行函数。
- 模型总结结果。

agentic tool use：

- 模型需要自己拆任务。
- 多工具、多轮调用。
- 工具结果会改变下一步计划。
- 失败时要重试、换策略或向用户澄清。
- 任务结束前要验证最终状态。

面试表达：

Function calling 是结构化输出能力，agentic tool use 是多步决策能力。真正难的是任务分解、状态管理、错误恢复、权限控制和结果验证。

Agent 系统结构

一个比较稳的 agent 系统通常包含：

模块	English	中文	作用
Controller	Controller	控制器	管理循环、预算、终止条件。

模块	English	中文	作用
Planner	Planner	规划器	拆解任务、选择工具。
Tool Runtime	Tool Runtime	工具运行时	执行搜索、代码、数据库、浏览器、文件操作。
Memory Store	Memory Store	记忆存储	保存长期偏好、任务状态、轨迹摘要。
Verifier	Verifier	验证器	检查测试、引用、状态、格式。
Policy Guard	Policy Guard	策略防护	权限、安全、隐私、速率限制。
Trace Logger	Trace Logger	轨迹日志	记录每次 tool call，便于 debug 和 eval。

不要在面试里说“我们用了 LangChain 所以是 agent”。框架不是核心，核心是任务闭环和评测闭环。

Evaluation：从 Model Eval 到 System Eval

LLM 应用不能只看 benchmark，需要分层评测。

层级	English	中文	例子
Model Eval	Model Evaluation	模型能力评测	MMLU、GPQA、AIME、HumanEval。
Retrieval Eval	Retrieval Evaluation	检索评测	recall@k、MRR、nDCG。
Generation Eval	Generation Evaluation	生成评测	factuality、faithfulness、format。
Tool Eval	Tool Evaluation	工具评测	tool call accuracy、参数准确率、错误恢复率。
Agent Eval	Agent Evaluation	智能体评测	SWE-bench、TAU-bench、BrowseComp、OSWorld。
System Eval	System Evaluation	系统评测	端到端成功率、延迟、成本、拒答率。
Safety Eval	Safety Evaluation	安全评测	jailbreak、PII 泄露、prompt injection、权限绕过。

面试表达：

LLM 项目评测要把“模型能力”和“系统能力”拆开。RAG 场景中，答案错可能来自 query rewrite、embedding、chunk、retrieval、rerank、context packing 或 generation。Agent 场景中，错误还可能来自 tool schema、状态管理、权限、预算和 verifier。

Prompt Injection 与 Tool Safety

Agent 接入工具后，安全问题会放大：

- 网页或文档里可能包含恶意指令。
- 检索内容可能要求模型泄露系统提示。
- 工具可能有写权限、删库权限、转账权限。
- 长任务中模型可能忘记原始约束。

基础防线：

1. 把用户指令、系统指令、工具输出分层。
2. 工具输出默认是不可信数据。
3. 高风险工具需要 human approval。
4. 每个工具做最小权限。
5. 对写操作做 dry-run 和 diff。
6. 记录 trace，便于审计。

本章面试高频问题

1. RAG 为什么能缓解幻觉？

RAG 把外部知识作为上下文提供给模型，让模型基于检索证据回答，而不是只依赖参数记忆。它还可以提供引用和可更新知识库。

2. RAG 的主要失败模式是什么？

主要包括检索不到、检索错、chunk 切坏、rerank 失败、上下文过长导致证据被忽略、文档冲突、权限过滤错误和生成阶段不忠实。

3. Agent 和普通 chat completion 有什么区别？

普通 chat completion 一次输入一次输出。Agent 多了工具调用、状态管理、观察反馈、多步决策、错误恢复和最终验证。

4. 怎么评估一个企业知识库问答系统？

需要评估检索 recall、答案 faithfulness、引用准确率、拒答策略、权限隔离、延迟、成本和人工标注通过率。

5. 怎么评估一个工具调用 agent？

不要只看最终回答。要看 tool selection accuracy、参数准确率、调用次数、成功率、错误恢复率、最终状态正确率、成本、延迟和安全违规率。

本章 References

- [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#)
- [Self-Consistency Improves Chain of Thought Reasoning in Language Models](#)
- [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)

- [REALM: Retrieval-Augmented Language Model Pre-Training](#)
- [ReAct: Synergizing Reasoning and Acting in Language Models](#)
- [Toolformer: Language Models Can Teach Themselves to Use Tools](#)
- [AgentBench: Evaluating LLMs as Agents](#)
- [TAU-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains](#)
- [BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents](#)
- [Holistic Evaluation of Language Models](#)
- [MTEB: Massive Text Embedding Benchmark](#)

第 4 章：VLM 基础、ViT、CLIP 与图文对齐

本章符号说明

符号/缩写	English	中文	含义
VLM	Vision-Language Model	视觉语言模型	同时建模视觉和语言的模型。
ViT	Vision Transformer	视觉 Transformer	把图像 patch 当作 token 输入 Transformer。
CLIP	Contrastive Language-Image Pre-training	对比式图文预训练	学习图像和文本共享 embedding 空间。
ITC	Image-Text Contrastive Loss	图文对比损失	拉近匹配图文，拉远不匹配图文。
VQA	Visual Question Answering	视觉问答	根据图像回答问题。
OCR	Optical Character Recognition	光学字符识别	识别图像中的文字。
I	Image	图像	输入图像。
T	Text	文本	输入文本或 caption。
z_I	Image Embedding	图像向量	图像编码器输出的全局表示。
z_T	Text Embedding	文本向量	文本编码器输出的全局表示。

本章目标

- 理解图像如何变成 token。
- 掌握 CLIP 的 contrastive learning 目标。
- 理解 zero-shot classification 和 image-text retrieval。
- 认识 VLM 常见任务和瓶颈。

ViT：图像变 token

ViT 把图像切成 patch：

$$I \rightarrow [p_1, p_2, \dots, p_n]$$

每个 patch 线性投影成 embedding，再加 position embedding，输入 Transformer encoder。

直觉：

- CNN 有强局部归纳偏置。
- ViT 更依赖数据规模和预训练。
- 对 VLM 来说，patch token 很自然地可以和 text token 对齐或融合。

CLIP

CLIP 同时训练 image encoder 和 text encoder。给定 batch 内 N 对图文样本：

$$(I_1, T_1), \dots, (I_N, T_N)$$

模型得到：

$$z_I = f_{img}(I), z_T = f_{text}(T)$$

相似度：

$$s_{ij} = z_{I_i}^T z_{T_j} / \tau$$

训练目标是让匹配的 (I_i, T_i) 相似度高，不匹配的 (I_i, T_j) 相似度低。

面试表达：

CLIP 的关键是用大规模图文对做 contrastive learning，把图像和文本投到同一个语义空间。可以通过文本 prompt 表示类别，实现 zero-shot image classification。

Zero-shot Classification

CLIP 做分类时，可以把类别写成 prompt：

```
a photo of a dog
a photo of a cat
```

然后计算图像 embedding 和每个文本 embedding 的相似度，选最高的类别。

优点：

- 不需要为每个新类别训练分类头。
- 类别可以由自然语言描述。
- 能做开放词表分类。

缺点：

- prompt 模板敏感。
- 对细粒度、计数、空间关系仍可能弱。
- 训练数据偏差会影响 zero-shot 表现。

VLM 常见任务

任务	English	中文	说明
Image Captioning	Image Captioning	图像描述	输入图像，输出自然语言描述。
VQA	Visual Question Answering	视觉问答	输入图像和问题，输出答案。
OCR QA	OCR Question Answering	文字识别问答	需要读取图像中的文字。
Chart QA	Chart Question Answering	图表问答	需要解析图表结构和数值。
Document QA	Document Question Answering	文档问答	需要理解版面、表格和文本。
Referring	Referring Expression	指代表达理解	根据文字定位图像中的对象。
Grounding	Visual Grounding	视觉定位	输出框、点或区域。

VLM 为什么难

VLM 不只是“把图片丢给 LLM”。

难点包括：

- 视觉 token 数量大，计算成本高。
- 图像包含空间、局部细节和文字信息。
- 文本 token 是离散符号，视觉 token 是连续空间表示。
- OCR、chart、document 需要精确读数和版面理解。
- 多图、多页、长视频带来上下文和时间建模问题。

本章面试高频问题

1. CLIP 为什么能 zero-shot?

CLIP 学到了图像和文本的共享语义空间。分类时把类别变成文本 prompt，比较图像与类别文本的相似度，就能不用训练分类头完成 zero-shot classification。

2. CLIP 和传统监督分类有什么区别？

传统分类学习固定 label space；CLIP 学图文对齐，label 可以由自然语言动态表达，因此更开放。

3. VLM 在文档理解上为什么难？

文档理解需要 OCR、版面结构、表格关系、阅读顺序和跨区域推理。单纯全局图像 embedding 往往不够，需要高分辨率、动态切图或专门的 document training data。

本章 References

- [An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale](#)

- [Learning Transferable Visual Models From Natural Language Supervision](#)
- [MMBench: Is Your Multi-modal Model an All-around Player?](#)
- [MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark](#)

第 5 章：MLLM 架构、训练流程与代表模型

本章符号说明

符号/缩写	English	中文	含义
MLLM	Multimodal Large Language Model	多模态大语言模型	在 LLM 上接入视觉等模态的模型。
BLIP-2	Bootstrapping Language-Image Pre-training 2	BLIP-2 模型	用 Q-Former 桥接 frozen image encoder 和 frozen LLM。
Flamingo	Flamingo VLM	Flamingo 视觉语言模型	用 gated cross-attention 接入视觉信息。
LLaVA	Large Language and Vision Assistant	大语言视觉助手	CLIP vision encoder + projection + LLM 的开源 VLM 路线。
Q-Former	Querying Transformer	查询 Transformer	BLIP-2 中从视觉特征抽取 query 表示的模块。
Projector	Projection Layer	投影层	把视觉特征映射到 LLM hidden size。
E_v	Vision Encoder	视觉编码器	把图像编码成视觉特征。
E_t	Text Encoder	文本编码器	把文本编码成语言表示。
H_v	Visual Hidden States	视觉隐状态	视觉 encoder 输出的 token 表示。

本章目标

- 掌握 VLM 接入 LLM 的主流方式。
- 理解 BLIP-2、Flamingo、LLaVA 的架构差异。
- 理解多模态预训练和 instruction tuning。
- 能解释现代 VLM 的动态分辨率和视频理解趋势。

VLM 接入 LLM 的三种思路

1. Late Fusion

先分别编码图像和文本，再在较高层融合。

优点是模块清晰，缺点是细粒度交互可能不足。

2. Cross-Attention

语言 token 在若干层 cross-attend 到视觉 token。Flamingo 是代表路线。

优点是视觉信息可在生成过程中被持续访问；缺点是架构改动较多。

3. Projector / Adapter

视觉 encoder 输出经过 projector 映射成 LLM 可读 token，再拼到文本 token 前后。LLaVA 是代表路线。

优点是简单、工程友好；缺点是视觉-语言对齐很依赖训练数据和 projector 能力。

BLIP-2

BLIP-2 的核心思想：冻结强大的 image encoder 和 LLM，只训练轻量 Q-Former 来桥接模态。

流程：

$$I \rightarrow E_v(I) \rightarrow QFormer(H_v) \rightarrow LLM$$

优点：

- 训练成本低。
- 利用已有视觉和语言模型能力。
- Q-Former 起到信息瓶颈和模态桥接作用。

Flamingo

Flamingo 支持 interleaved image-text 输入，即文本和多张图像交错出现。核心包括：

- frozen vision encoder。
- frozen language model。
- perceiver resampler。
- gated cross-attention。

面试表达：

Flamingo 的重点是让 LLM 在生成时通过 cross-attention 访问视觉特征，并能处理多图交错上下文，因此在 few-shot 多模态任务上很有代表性。

LLaVA

LLaVA 的典型结构：

$$Image \rightarrow CLIP\ Encoder \rightarrow Projector \rightarrow LLM$$

训练通常分两阶段：

1. Pre-alignment：冻结大部分模块，只训练 projector，让视觉 token 对齐语言空间。
2. Visual instruction tuning：用图文指令数据训练模型对话能力。

优点：

- 架构简单。
- 复现成本相对低。
- 很适合面试中说明 open-source MLLM pipeline。

Qwen2.5-VL 代表的新趋势

现代 VLM 不只做单图问答，还强调：

- dynamic resolution。
- document parsing。
- grounding with box/point。
- long-video understanding。
- agentic computer/mobile 操作。
- OCR、chart、table 的结构化抽取。

面试表达：

早期 VLM 更像图像 caption/VQA 模型，现代 MLLM 更像通用视觉 agent：能读文档、看视频、定位对象、调用工具并完成任务。

训练数据

常见数据阶段：

阶段	English	中文	作用
Image-Text Pretraining	Image-Text Pretraining	图文预训练	学习基础图文对齐。
Caption Data	Caption Data	图像描述数据	学会描述视觉内容。
VQA Data	Visual QA Data	视觉问答数据	学会根据问题聚焦视觉信息。
OCR/Doc Data	OCR/Document Data	OCR/文档数据	学会读文字、表格、版面。
Instruction Data	Instruction Data	指令数据	学会对话格式和复杂任务。
Preference Data	Preference Data	偏好数据	改善回答质量、安全和风格。

本章面试高频问题

1. BLIP-2 为什么要冻结 image encoder 和 LLM?

冻结可以显著降低训练成本，同时复用强视觉模型和强语言模型的能力。Q-Former 作为桥接模块学习如何把视觉信息转成 LLM 能理解的表示。

2. Flamingo 和 LLaVA 的主要区别?

Flamingo 通过 gated cross-attention 在语言模型层内接入视觉信息，支持 interleaved image-text few-shot。LLaVA 更简单，用 CLIP encoder 加 projector 把视觉 token 映射到 LLM 输入空间。

3. VLM 为什么需要 instruction tuning?

图文预训练让模型学会对齐视觉和文本，但不一定会遵循用户问题、输出格式或多轮对话。instruction tuning 让模型学会以助手形式完成多模态任务。

4. 高分辨率图像为什么难?

图像分辨率越高，patch token 越多，attention 成本越高。需要动态分辨率、裁剪、多尺度、窗口 attention 或视觉 token 压缩。

本章 References

- [Flamingo: a Visual Language Model for Few-Shot Learning](#)
- [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#)
- [Visual Instruction Tuning](#)
- [Qwen2.5-VL Technical Report](#)
- [GPT-4 Technical Report](#)

第 6 章：VLM 数据构造、预处理与训练实操

这一章把 VLM（Vision-Language Model，视觉语言模型）从“模型结构”推进到“怎么训”。面试里如果只说 CLIP、LLaVA、Qwen2.5-VL，很容易停在概念层；真正能拉开差距的是说明多模态数据如何构造、清洗、打标、打包，以及每个训练阶段解决什么问题。

本章符号说明

符号/缩写	English	中文	含义
VLM	Vision-Language Model	视觉语言模型	同时处理视觉和语言输入的模型。
MLLM	Multimodal Large Language Model	多模态大语言模型	把图像、视频、音频等模态接入 LLM 的模型族。
LLM	Large Language Model	大语言模型	多模态模型中负责语言理解、推理和生成的核心。
OCR	Optical Character Recognition	光学字符识别	从图像、截图、扫描件中识别文字。
VQA	Visual Question Answering	视觉问答	根据图像或视频回答问题。
SFT	Supervised Fine-Tuning	监督微调	用指令-答案数据训练模型遵循任务。
DPO	Direct Preference Optimization	直接偏好优化	用偏好对直接优化模型输出偏好。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward，再优化模型。
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	用自动可验证的任务结果作为奖励信号。
ETL	Extract, Transform, Load	抽取、转换、加载	数据工程里的标准处理流程。
JSONL	JSON Lines	JSON 行格式	每行一个 JSON 样本，常用于训练数据。
DPI	Dots Per Inch	每英寸点数	文档扫描件和截图清晰度指标。
ROI	Region of Interest	感兴趣区域	图像中需要重点理解或裁剪的区域。
ViT	Vision Transformer	视觉 Transformer	把图像切成 patch token 的视觉编码器。
CLIP	Contrastive Language-Image Pre-training	对比式图文预训练	学习图文共享语义空间的经典模型。

符号/缩写	English	中文	含义
I	Image	图像	输入的单张图像。
V	Video	视频	输入的视频片段或帧序列。
T_v	Visual Tokens	视觉 token	图像或视频经 encoder/tokenizer 后得到的 token。
T_x	Text Tokens	文本 token	prompt、问题、答案或系统指令的 token。
N_v	Number of Visual Tokens	视觉 token 数量	决定上下文占用和 attention 成本。
D	Dataset	数据集	多模态训练样本集合。
q	Question	问题	VQA、文档问答或视觉推理中的用户问题。
a	Answer	答案	模型需要生成的目标回答。

本章目标

- 能讲清 VLM 数据从原始资产到训练样本的完整 pipeline。
- 能区分 image-text pretraining、instruction tuning、preference tuning、RLVR 数据。
- 能解释 dynamic resolution、tiling、OCR、document parsing 为什么重要。
- 能设计一个 VLM 项目的数据 schema、质量过滤和训练阶段。

VLM 数据为什么要单独拆

LLM 的数据主要是 token 序列，VLM 的数据同时包含视觉内容、文本描述、布局、时间、坐标和任务格式。问题不只是“有没有图”，而是：

- 图像是否清晰、完整、无水印、无隐私风险。
- caption 是否描述了细粒度属性、关系、文字和动作。
- OCR 文本是否和图像坐标、版面结构对齐。
- 高分辨率图像如何压进有限 visual token budget。
- 文档、图表、UI、视频是否有适合任务的标注。
- 训练样本是否覆盖模型上线后的真实问题分布。

面试表达：

VLM 的训练效果很大一部分来自数据工程。视觉输入不是普通文本 token，必须先处理分辨率、裁剪、版面、OCR、caption、坐标和任务 schema，否则模型很容易在 OCR、图表、空间关系和细粒度 grounding 上掉链子。

数据类型地图

数据类型	English	中文	典型用途	常见风险
Image-Text Pair	Image-Text Pair	图文对	图文对齐、caption、检索	caption 粗糙、噪声大、版权不清
Interleaved Data	Interleaved Image-Text Data	图文交错数据	多图理解、上下文学习	顺序错乱、图文引用不清
VQA Data	Visual Question Answering Data	视觉问答数据	视觉问答、细粒度理解	答案过短、问题模板化解
OCR Data	Optical Character Recognition Data	文字识别数据	文档、票据、截图、UI	低 DPI、文字遮挡、标注错
Document Data	Document Understanding Data	文档理解数据	PDF、表格、版面解析	结构丢失、页码和区域不对齐
Chart Data	Chart and Table Data	图表数据	图表问答、数值读取	数字精度、坐标轴语义错误
Grounding Data	Grounding Data	定位数据	box、point、region QA	坐标归一化错误
Video Data	Video Understanding Data	视频理解数据	动作、事件、时间推理	采样稀疏、caption 不含时间变化
UI Data	User Interface Data	界面数据	computer-use、GUI agent	点击坐标、状态变化难标注
Preference Data	Preference Data	偏好数据	回答质量、安全、格式	judge 偏差、偏好维度混淆

从原始数据到训练样本

一个 VLM 数据 pipeline 可以这样讲：

原始图片 / 视频 / PDF / 截图

→ 去重、质量过滤、安全过滤、版权和隐私过滤

→ OCR / layout parsing / object detection / captioning / recaptioning

→ 任务构造: caption、VQA、doc QA、chart QA、grounding、UI action

→ schema 统一: messages、images、boxes、metadata、source、license

→ train / dev / test 切分与去污染

→ packing、resolution bucket、visual token budget 控制

→ SFT / preference / RLVR / eval harness

这里每一步都可能决定模型上线质量。比如 OCR 数据如果没有保留坐标，模型可能“读字”，但不能回答“右上角表格第二列是多少”；视频 caption 如果只写“一个人在走路”，模型很难学到镜头、动作变化和事件顺序。

图像预处理：分辨率、裁剪和 visual token

图像进入 VLM 前通常会被切成 patch 或由 vision encoder 编码。对于 ViT 类模型，粗略可以理解成：

$$N_v \approx H/p \times W/p$$

其中 H (Height, 高度) 和 W (Width, 宽度) 是图像尺寸, p (Patch Size, patch 尺寸) 是每个视觉 patch 的边长, N_v (Number of Visual Tokens, 视觉 token 数量) 决定计算成本。

常见策略：

策略	English	中文	适合场景	代价
Resize	Resize	缩放	普通自然图像	小字、细节丢失
Center Crop	Center Crop	中心裁剪	主体居中图片	边缘信息丢失
Dynamic Resolution	Dynamic Resolution	动态分辨率	多宽高比、多任务	训练和 batch 更复杂
Tiling	Image Tiling	图像切块	文档、长图、截图	token 数多, 跨 tile 关系难
ROI Crop	Region of Interest Crop	重点区域裁剪	OCR、商品、图表	依赖检测或规则
Multi-scale	Multi-scale Encoding	多尺度编码	细节和全局都重要	推理成本高

面试表达：

高分辨率 VLM 不是简单把图片放大。真正的问题是如何在有限 visual token budget 里同时保留全局布局和局部细节，所以 dynamic resolution、tiling、ROI crop 和多尺度策略会直接影响 OCR、文档和图表能力。

Caption 与 Recaption

早期图文对 caption 很粗，例如“a dog on grass”。现代 VLM 训练更需要 rich caption：

- 主体、属性、数量、颜色、材质。
- 空间关系和相对位置。
- OCR 文本和可见标志。
- 图表坐标轴、单位、趋势。
- 文档版面、标题、表格、脚注。
- 视频里的动作、镜头、时间顺序。
- 不确定性和遮挡说明。

常用做法是用强 VLM 或人工标注生成 recaption，再用规则和 judge 过滤低质量描述。

高质量 caption 不是越长越好，而是要和任务相关。如果目标是文档 QA，就要保留版面、表格结构和 OCR；如果目标是商品理解，就要保留品牌、类目、材质、缺陷和使用场景。

训练样本 Schema

一个工程上常用的 VLM SFT 样本可以长这样：

```
{
  "id": "docqa_000001",
  "images": ["page_001.png"],
  "messages": [
    {"role": "system", "content": "You are a visual document assistant."},
    {"role": "user", "content": "根据图片中的表格, 2025 Q4 的收入是多少? "},
    {"role": "assistant", "content": "2025 Q4 的收入是 1280 万元。"}
  ],
  "metadata": {
    "task": "document_qa",
    "source": "internal_synthetic",
    "ocr_available": true,
    "answer_type": "numeric"
  }
}
```

带 grounding 的样本还要记录坐标：

```
{
  "id": "grounding_000017",
  "images": ["shelf.png"],
  "messages": [
    {"role": "user", "content": "请指出红色瓶装饮料的位置。"},
    {"role": "assistant", "content": "<box>0.62,0.31,0.78,0.67</box>"}
  ],
  "metadata": {
    "task": "grounding",
    "coordinate": "normalized_xyxy"
  }
}
```

注意两点：

- schema 要稳定，否则训练和评测难复现。
- 坐标格式必须明确，例如 normalized xyxy、pixel xyxy、center-width-height，混用会造成灾难性错误。

训练阶段怎么拆

阶段	English	中文	主要数据	目标
Vision Encoder Pretraining	Vision Encoder Pretraining	视觉编码器预训练	图像、图文对	获得视觉表征
Projector Alignment	Projector Alignment	投影层对齐	image-caption	让视觉 token 进入 LLM 语义空间
Multimodal Pretraining	Multimodal Pretraining	多模态预训练	大规模图文/图文交错	学习跨模态对齐和基础生成
Instruction Tuning	Instruction Tuning	指令微调	VQA、doc QA、OCR、grounding	学会按用户问题回答
Preference Tuning	Preference Tuning	偏好调优	pairwise response	改善有用性、格式、安全和拒答
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	OCR、数学图表、UI 状态、代码截图	用自动判定提升可验证任务
Distillation	Knowledge Distillation	知识蒸馏	teacher 输出、轨迹	降低成本、提升小模型能力

面试里可以说：

VLM 训练通常不是一步到位。先解决视觉-语言对齐，再解决指令遵循，再针对 OCR、文档、图表、grounding、视频和 agent 任务做专门数据，最后用偏好或可验证奖励修正输出质量。

VLM 数据质量检查清单

检查项	说明
去重	同图、近似图、视频帧重复会让训练和评测虚高。
去污染	benchmark、dev/test、线上回归集不能泄漏到训练集。
图像质量	过滤过低分辨率、严重模糊、遮挡、水印、损坏文件。
OCR 质量	检查文字框、读取顺序、表格结构和单位。
Caption 质量	检查是否覆盖主体、属性、关系、文字、动作。
安全和隐私	过滤人脸、身份证、手机号、医疗、未成年人等敏感内容。
版权和来源	记录 source、license、爬取时间、授权范围。
分布覆盖	覆盖目标业务场景，不要只优化公开 benchmark。
Schema 一致	坐标、messages、图片路径、metadata 字段必须稳定。

实操项目：企业文档 VLM

如果你要在面试里讲一个 VLM 项目，可以用企业文档 QA：

1. 数据：PDF、扫描件、表格、票据、PPT 截图。
2. 预处理：PDF render、OCR、layout parsing、table extraction、page image。
3. 训练：doc caption、page QA、table QA、区域定位、拒答样本。
4. 评测：DocVQA、TextVQA、ChartQA、自建业务集。
5. 指标：exact match、numeric tolerance、citation accuracy、OCR error attribution。
6. 风险：权限、PII、表格错读、跨页引用、幻觉引用。

一句话版本：

我不会把企业文档 VLM 做成单纯图片问答，而是会同时建 OCR/layout 结构、page image、检索和可验证问答数据，让模型既能看版面，也能输出可追溯答案。

本章面试高频问题

1. VLM 数据和 LLM 数据最大的区别？

VLM 数据需要处理视觉质量、分辨率、版面、OCR、坐标、时间和多模态 schema。LLM 数据主要是文本序列，VLM 数据还要保证图文对齐、区域对齐和任务对齐。

2. 为什么 VLM 容易在 OCR 和图表上出错？

OCR 和图表要求细粒度视觉识别、布局理解、数值读取和推理。普通 caption/VQA 数据不一定训练这些能力，高分辨率处理和结构化标注不足时，模型容易看错字、读错轴、算错数。

3. Dynamic resolution 有什么价值？

它允许模型根据图片宽高比和任务复杂度使用不同视觉 token 数，在成本可控的情况下保留更多细节，尤其对文档、长图、截图、图表很重要。

4. VLM 为什么需要 preference 或 RLVR？

SFT 能让模型会回答，但不一定可靠。preference 可以优化回答风格、拒答和安全；RLVR 可以在 OCR、表格数值、UI 状态、数学图形等可验证任务上提供更强学习信号。

本章 References

- [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#)
- [Visual Instruction Tuning](#)
- [Qwen2.5-VL Technical Report](#)
- [InternVL 2.5: Expanding Performance Boundaries of Open-Source Multimodal Models](#)
- [DocVQA: A Dataset for VQA on Document Images](#)
- [ChartQA: A Benchmark for Question Answering about Charts](#)

第 7 章：VLM 评测 Harness、Benchmark 与项目实操

VLM（Vision-Language Model，视觉语言模型）的评测不能只看一个总分。一个模型可能 MMMU 很高，但 OCR 错；可能文档 QA 好，但空间关系差；可能单图强，但视频理解弱。面试里要把评测讲成 harness：固定数据、输入、模型版本、prompt、采样参数、工具、指标、judge 和 badcase 归因。

本章符号说明

符号/缩写	English	中文	含义
VLM	Vision-Language Model	视觉语言模型	同时处理视觉和语言输入的模型。
MLLM	Multimodal Large Language Model	多模态大语言模型	统一处理文本、图像、视频等模态的大模型。
Eval	Evaluation	评测	对模型能力、风险、成本和稳定性的测量。
Harness	Evaluation Harness	评测框架	统一管理数据、prompt、运行、打分、报告和回归的系统。
Benchmark	Benchmark	基准测试	固定任务集合和指标，用于横向比较。
Judge	Judge Model	裁判模型	对开放式回答打分或比较的模型。
Oracle	Oracle	标准判定器	可可靠判断答案或状态是否正确的程序或标注。
Trace	Trace	轨迹日志	记录输入、输出、中间步骤、工具调用和分数。
OCR	Optical Character Recognition	光学字符识别	识别图片、文档、截图中的文字。
VQA	Visual Question Answering	视觉问答	根据视觉内容回答问题。
DocVQA	Document Visual Question Answering	文档视觉问答	针对文档图片的问题回答任务。
ChartQA	Chart Question Answering	图表问答	针对图表内容的问题回答任务。
TextVQA	Text Visual Question Answering	文字视觉问答	需要读取图中文字的 VQA 任务。

符号/缩写	English	中文	含义
MMMU	Massive Multi-discipline Multimodal Understanding	大规模多学科多模态理解基准	专家级多学科 VLM 评测。
MME	Multimodal Evaluation Benchmark	多模态评测基准	评估感知和认知能力的 VLM benchmark。
VHELM	Vision Holistic Evaluation of Language Models	视觉语言模型整体评测	HELM 思路扩展到 VLM 的综合评测。
UI	User Interface	用户界面	GUI/computer-use agent 需要理解和操作的界面。
GUI	Graphical User Interface	图形用户界面	包含按钮、文本、布局、状态的可视化界面。
IoU	Intersection over Union	交并比	目标定位框重叠程度指标。
EM	Exact Match	精确匹配	预测答案和标准答案完全一致。
M	Model	模型	被评测的 VLM 或系统。
T	Task Set	任务集合	评测样本集合。
s_i	Score	样本分数	第 i 个样本的得分。

本章目标

- 能设计 VLM eval harness，而不是只引用 benchmark 分数。
- 能按 OCR、文档、图表、空间、视频、GUI、推理拆分能力。
- 能说明 VLM judge 的优缺点和校准方式。
- 能把 VLM 项目讲成数据、模型、评测、上线闭环。

VLM 评测为什么容易误导

VLM 评测有几个常见陷阱：

- 单图 benchmark 高，不代表文档、图表、视频、UI 强。
- 多选题高，不代表开放式回答可靠。
- OCR benchmark 高，不代表能做跨区域、跨页、表格推理。
- VLM judge 打分高，不代表事实正确。
- 图片经过压缩、裁剪或 resize 后，模型能力会明显变化。
- prompt、resolution、max tokens、工具权限不同，benchmark 不可比。

面试表达：

VLM 的核心不是报一个总分，而是按任务维度拆能力，再用固定 harness 控制输入分辨率、prompt、模型版本、工具和打分方式。否则很容易把模型能力、数据处理能力和评测脚手架混在一起。

能力矩阵

能力	English	中文	典型数据	指标
Perception	Visual Perception	视觉感知	MME、POPE	accuracy、F1
OCR	Optical Character Recognition	文字识别	TextVQA、OCRBench	EM、edit distance
Document QA	Document Question Answering	文档问答	DocVQA、自建 PDF 集	EM、numeric tolerance、citation
Chart QA	Chart Question Answering	图表问答	ChartQA、PlotQA	accuracy、numeric tolerance
Spatial Reasoning	Spatial Reasoning	空间推理	图形、几何、关系题	accuracy、reasoning judge
Grounding	Visual Grounding	视觉定位	RefCOCO、box/point 数据	IoU、point accuracy
Video Understanding	Video Understanding	视频理解	Video-MME、MVBench	accuracy、temporal consistency
GUI Understanding	User Interface Understanding	界面理解	OSWorld、UI 任务集	task success、click accuracy
Safety	Safety Evaluation	安全评测	红队集、隐私集	violation rate、refusal quality

一个 VLM Eval Harness 长什么样

Dataset Registry

- Input Renderer: 图片、PDF 页面、视频帧、UI 截图
- Prompt Builder: 固定系统指令、问题模板、输出格式
- Model Adapter: OpenAI / Gemini / Qwen / GLM / local model
- Runner: 并发、重试、缓存、seed、版本记录
- Scorer: EM、F1、IoU、numeric tolerance、judge、human review
- Error Analyzer: OCR、视觉感知、推理、格式、拒答、幻觉归因
- Report: 总分、分场景、badcase、成本、延迟、回归趋势

最重要的是 trace。每个样本至少记录：

- 原始图片或视频 hash。

- 输入分辨率、裁剪、tile 配置。
- prompt 和输出格式。
- 模型名称、版本、temperature、max tokens。
- OCR、检索、工具调用等中间结果。
- 标准答案、自动分数、人评分数。
- badcase 标签。

没有 trace，就很难解释“为什么这次模型变差了”。

VLM Judge 的使用边界

VLM judge 可以提高开放式评测效率，但不能无脑相信。

适合：

- 图像描述质量。
- 多答案开放题。
- 参考一致性。
- 安全和拒答质量。
- 复杂项目的 pairwise 比较。

不适合单独承担：

- 数字精确读取。
- OCR 字符级正确性。
- 坐标定位。
- 业务规则判定。
- 权限和隐私合规。

建议做法：

任务	首选打分	辅助打分
OCR	edit distance、EM	judge 解释错误类型
表格数值	numeric tolerance	judge 归因
Grounding	IoU、point accuracy	人工抽检
开放描述	pairwise judge	人评校准
安全	rule + classifier	judge + 人评
GUI agent	final state oracle	trace review

Badcase 归因

VLM 项目里，badcase 不能只写“回答错了”。建议至少分成：

Badcase 类型	中文	说明
Perception Error	感知错误	没看清主体、颜色、数量、位置。
OCR Error	文字识别错误	看错字、漏字、读序错误。
Layout Error	版面错误	表格、列、标题、跨页关系错。
Reasoning Error	推理错误	视觉读对了，但推理或计算错。
Grounding Error	定位错误	box、point、区域对应错。
Format Error	格式错误	没按 JSON、表格或字段要求输出。
Refusal Error	拒答错误	该答不答或不该答却答。
Hallucination	幻觉	生成图中没有的内容或引用不存在证据。
Tool Error	工具错误	OCR、检索、浏览器、GUI 工具返回错误或使用不当。

这个标签体系能直接反推下一步改数据、改模型、改预处理还是改 harness。

项目实操一：文档理解助手

面试讲法：

1. 任务：企业 PDF、扫描件、报表、合同问答。
2. 数据：真实脱敏文档、合成文档、公开 DocVQA/ChartQA、业务高频问题。
3. 预处理：PDF render、OCR、layout parsing、table extraction、page-level image。
4. 模型：VLM + RAG，长文档先检索页，再让 VLM 看对应页面。
5. 评测：answer EM、numeric tolerance、citation precision、page recall、拒答准确率。
6. 风险：权限、PII、跨页引用、表格错读、过期文档。

亮点表达：

文档 VLM 不应该让模型盲看整份 PDF。更稳的做法是把检索、OCR/layout 和 VLM 结合起来：先定位页和区域，再让 VLM 基于证据回答，并把 citation 和 page id 作为评测对象。

项目实操二：视觉质检和商品理解

适合电商、广告、内容审核场景：

模块	说明
数据	商品主图、详情图、用户上传图、质检标签。
任务	类目识别、属性抽取、缺陷检测、敏感元素检测。
模型	VLM 做开放理解，检测器做硬约束。

模块	说明
评测	属性 F1、缺陷 recall、误杀率、人工复核一致性。
上线	高风险样本进入人工审核，低风险样本自动通过。

面试要强调：VLM 适合开放语义，硬安全和精确定位最好结合专用 detector、OCR、规则和人工复核。

项目实操三：GUI / Computer-use Agent

VLM 在 GUI agent 里的作用是读界面、定位控件、理解状态变化。

一个可复现评测要记录：

- 初始网页或 app 状态。
- 用户目标。
- 每一步 screenshot。
- 模型动作，例如 click、type、scroll。
- 工具 observation。
- 最终状态 oracle。
- 是否触发权限或安全边界。

指标不要只看“模型说完成了”，而要看最终状态是否真的满足目标。

面试高频问题

1. VLM benchmark 怎么选？

先按业务能力拆分：OCR、文档、图表、空间、视频、GUI、安全。公开 benchmark 用来横向比较，自建 regression set 用来覆盖真实业务。二者都要有，不能只跑 leaderboard。

2. 为什么 VLM judge 需要校准？

VLM judge 自身也会 hallucinate，对风格、长度和模型家族有偏好。需要人工标注集校准，报告 judge-human agreement，并对精确任务使用程序化 oracle。

3. VLM 项目怎么证明有效？

用离线 benchmark 证明基础能力，用业务回归集证明场景覆盖，用线上 A/B 或人工复核指标证明收益。还要报告成本、延迟、失败类型和安全边界。

4. VLM 和 OCR/layout/RAG 怎么配合？

对于文档和企业知识库，VLM 不一定替代 OCR 和 RAG。更稳的系统通常是 OCR/layout 提供结构，RAG 定位证据，VLM 负责跨模态理解和回答，最后用 citation 和 oracle 评测。

本章 References

- [MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark](#)
- [MME: A Comprehensive Evaluation Benchmark for Multimodal Large Language Models](#)
- [VHELM: A Holistic Evaluation of Vision Language Models](#)

- [DocVQA: A Dataset for VQA on Document Images](#)
- [ChartQA: A Benchmark for Question Answering about Charts](#)
- [Video-MME: The First-Ever Comprehensive Evaluation Benchmark of Multi-modal LLMs in Video Analysis](#)
- [OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments](#)

第 8 章：LLM/VLM 进阶面试速查与项目表达

高频概念横向对比

概念	English	中文	面试一句话
Transformer	Transformer Architecture	Transformer 架构	用 attention 并行建模 token 间依赖。
Pretraining	Pretraining	预训练	大规模 next-token prediction 学语言、代码、知识和分布。
SFT	Supervised Fine-Tuning	监督微调	用指令数据让模型学会按任务格式输出。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用偏好训练 reward，再优化策略。
RLAIF	Reinforcement Learning from AI Feedback	基于 AI 反馈的强化学习	用 AI judge 或 constitution 降低人类反馈成本。
DPO	Direct Preference Optimization	直接偏好优化	把偏好学习变成分类式 loss，省掉 RL loop。
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	用测试、答案校验、工具状态等自动 reward 训练。
GRPO	Group Relative Policy Optimization	组相对策略优化	用同组样本相对 reward 构造 advantage，省掉 value model。
PRM	Process Reward Model	过程奖励模型	给中间推理步骤打分。
ORM	Outcome Reward Model	结果奖励模型	只给最终结果打分。
LoRA	Low-Rank Adaptation	低秩适配	冻结原权重，只训练低秩更新。
RAG	Retrieval-Augmented Generation	检索增强生成	检索证据后生成，增强可更新性和溯源。
Hybrid Search	Hybrid Search	混合检索	组合 BM25、dense retrieval 和 metadata filter。
GraphRAG	Graph Retrieval-Augmented Generation	图检索增强生成	用实体关系和图结构支持多跳、全局主题类问题。
RAGAS	RAG Assessment	RAG 评估工具	自动评估 RAG faithfulness、context relevance 等指标。
Agent	Agent	智能体	在 observe-plan-act-verify 循环中调用工具解决任务。

概念	English	中文	面试一句话
Harness	Harness	评测/执行框架	管理数据、prompt、工具、环境、打分和 trace 的系统层。
Benchmark	Benchmark	基准测试	固定任务集合和指标，用于比较模型或系统。
MoE	Mixture of Experts	混合专家模型	每个 token 只激活部分专家，提升容量/成本比。
CLIP	Contrastive Language-Image Pre-training	对比式图文预训练	用对比学习把图像和文本映射到共享语义空间。
LLaVA	Large Language and Vision Assistant	大语言视觉助手	CLIP vision encoder + projector + LLM 的多模态指令路线。

高频问答

1. 为什么 LLM 会 hallucination?

因为预训练目标是拟合 token 分布，不是事实数据库查询；模型会生成高概率、语义连贯但不一定真实的文本。知识过期、上下文不足、问题诱导、检索错误都会加剧 hallucination。

2. 怎么降低 hallucination?

常用方法包括 RAG、引用约束、答案验证、拒答策略、tool use、结构化输出校验、领域 SFT、偏好优化、RLVR 和人工评测闭环。

3. RAG 和 fine-tuning 怎么选?

RAG 适合频繁更新、需要溯源的知识；fine-tuning 适合格式、风格、领域术语、任务习惯。知识注入优先 RAG，行为模式优先 fine-tuning。

4. RLHF、DPO、RLVR 怎么区分?

RLHF 用 reward model + RL 优化偏好；DPO 直接用偏好对优化，稳定但更偏离线；RLVR 用可验证 reward，适合数学、代码、工具调用和 agent 任务。

5. GRPO 为什么火?

GRPO 省掉 value model，用同组样本的相对 reward 估计 advantage，适合大模型 reasoning RL。但它依赖 reward 分布和采样质量，遇到全对/全错组时学习信号会弱。

6. Agent 和 function calling 的区别?

Function calling 是一次或少数几次结构化调用。Agent 是多步闭环：规划、调用工具、观察结果、调整计划、验证最终状态。

7. Harness、benchmark、scaffold 怎么区分?

Benchmark 是题和指标，harness 是执行评测的系统层，scaffold 是 agent 外层的规划、工具、记忆、验证和重试逻辑。2026 年比较 agent 时要报告 model + harness + scaffold + budget，而不是只报模型名。

8. 为什么 coding agent 难?

它要理解仓库、定位 bug、编辑多个文件、运行测试、读失败日志、持续修正，并且避免过度修改。这是 long-horizon task，不是代码补全。

9. VLM 项目怎么讲?

按“任务-数据-模型-训练-评测-风险”讲。例如票据理解：任务是 OCR + table parsing + field extraction；数据是扫描件和字段标签；模型用 VLM 或 OCR+LLM pipeline；指标包括字段准确率、表格结构 F1、拒答率和人工复核成本。

10. 如何评价一个 VLM?

不要只报单个 benchmark。要分 OCR、caption、VQA、chart、document、grounding、multi-image、long-video、robustness 和 safety。

11. 2026 年 SOTA 大模型路线怎么总结?

主线是 reasoning + tool use + verifiable RL + long-horizon agent。闭源模型强调系统化、router、safe-completions 和产品安全；开源模型强调 MoE、RLVR、thinking budget、agentic data 和蒸馏。

项目 1：企业知识库 RAG

任务目标：

- 支持公司内部文档问答。
- 输出答案和引用。
- 对无证据问题拒答。

技术方案：

1. 文档解析：PDF、Markdown、网页、表格。
2. Chunking：按标题、段落、表格结构切分。
3. Embedding：选择中文和英文表现稳定的 embedding model。
4. Retrieval：向量召回 + BM25 混合召回。
5. Rerank：cross-encoder 或 LLM rerank。
6. Generation：答案必须引用来源。
7. Verification：引用是否支持答案，是否越权。
8. Eval：人工 gold set + 自动一致性检查。

面试亮点：

- 你能区分 retrieval error 和 generation error。
- 你设计了拒答和引用验证。
- 你关注权限隔离和数据新鲜度。
- 你能说明 RAG harness：corpus version、index version、gold evidence、permission scope、citation scorer。

项目 2：Coding Agent

任务目标：

- 输入 issue 或用户需求。
- 自动定位代码、修改、运行测试并输出 diff。

技术方案：

1. Repository indexing：符号索引、全文搜索、依赖图。
2. Planning：先读问题和相关测试，再做最小修改计划。
3. Editing：限制文件范围，生成 patch。
4. Verification：运行单测、lint、类型检查、回归测试。
5. Repair loop：根据失败日志迭代。
6. Review：输出风险、验证命令、未覆盖点。

指标：

- issue resolution rate。
- test pass rate。
- regression rate。
- tool error rate。
- tokens / task。
- human review time reduction。
- trace replay success rate。

面试亮点：

我不会只让模型“写代码”，而是把代码搜索、编辑、测试、日志分析、回归验证和最小权限放进闭环。最终以测试和 diff 证明任务完成。

项目 2.5：Eval Harness / Agent Harness

任务目标：

- 为 LLM/RAG/agent 系统建立可复现评测。
- 固定数据、prompt、模型版本、工具权限、预算和打分协议。
- 输出 trace、错误分类和回归报告。

技术方案：

1. Dataset adapter：读取样本、版本和 gold answer。
2. Model adapter：统一不同 API 和本地模型。
3. Task adapter：构造 prompt 或 agent 环境。
4. Runner：控制 batch、timeout、temperature、reasoning effort。
5. Scorer：规则、oracle、LLM-as-a-judge、人工抽检。
6. Trace logger：记录 prompt、tool call、observation、cost、latency。
7. Report：按 failure type 聚合。

指标：

- pass@1 / pass@k。
- exact match / F1。
- judge agreement。
- cost / latency。
- tool error rate。
- permission violation rate。
- reproducibility rate。

项目 3：多模态文档理解

任务目标：

- 输入发票、报告、截图或表格图片。
- 输出结构化 JSON。

技术方案：

1. OCR 或 VLM 读取文字。
2. Layout-aware chunking。
3. 字段抽取和 schema validation。
4. 对低置信字段触发人工复核。

指标：

- field accuracy。
- exact match。
- table F1。
- hallucinated field rate。
- human review reduction。

项目 4：Agentic Research / Deep Research

任务目标：

- 输入一个复杂研究问题。
- 自动搜索、阅读、交叉验证、生成带引用报告。

技术方案：

1. Query decomposition。
2. Web search / paper search。
3. Source credibility scoring。
4. Evidence table。
5. Draft generation。
6. Citation verification。
7. Contradiction check。

指标：

- answer correctness。
- citation precision。
- source diversity。
- unsupported claim rate。
- time-to-answer。
- human edit distance。

45 分钟复习路线

1. 5 分钟：Transformer、attention、causal mask。
2. 5 分钟：pretraining、SFT、RLHF、DPO。
3. 5 分钟：RLVR、GRPO、PRM/ORM、reasoning model。
4. 5 分钟：LoRA/QLoRA、MoE、推理优化、KV cache。
5. 5 分钟：RAG pipeline、hybrid retrieval、GraphRAG、RAG eval。
6. 5 分钟：agent loop、tool use、coding agent、harness。
7. 5 分钟：CLIP、BLIP-2、LLaVA、Qwen2.5-VL。
8. 5 分钟：GPT-5.5、Claude Opus 4.7、Gemini 2.5、DeepSeek-R1、Qwen3、Kimi K2、GLM-5.2。
9. 5 分钟：项目讲法和评测指标。

最后检查清单

- 能手写 attention 公式。
- 能解释为什么 next-token prediction 能学到通用能力。
- 能讲清 SFT、RLHF、DPO、RLVR 的区别。
- 能讲清 GRPO 为什么省 value model。
- 能讲清 PRM/ORM 的优缺点。
- 能讲清 RAG 失败模式。
- 能讲清 RAG harness 和 citation verification。
- 能讲清 agentic tool use 的状态、工具、权限、验证。
- 能讲清 harness、benchmark、scaffold、leaderboard 的区别。
- 能讲清 coding agent 和代码补全的差别。
- 能讲清 CLIP 的 contrastive loss 直觉。
- 能讲清 LLaVA/BLIP-2/Flamingo 的差异。
- 能给出一个完整 LLM/VLM 项目方案。

第 9 章：Reasoning Model、RLVR 与过程监督

本章符号说明

符号/缩写	English	中文	含义
Reasoning Model	Reasoning Model	推理模型	会在回答前分配额外推理计算的模型。
Test-time Compute	Test-time Compute	测试时计算	推理阶段额外使用的 token、采样、搜索或工具调用预算。
CoT	Chain-of-Thought	思维链	中间推理过程。
Hidden CoT	Hidden Chain-of-Thought	隐藏思维链	不直接展示给用户的内部推理轨迹。
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	用可自动验证的 reward 训练模型。
PRM	Process Reward Model	过程奖励模型	对中间步骤打分。
ORM	Outcome Reward Model	结果奖励模型	对最终答案或最终状态打分。
GRPO	Group Relative Policy Optimization	组相对策略优化	用组内相对 reward 训练策略的 RL 算法。
Best-of-N	Best-of-N Sampling	N 选 1 采样	采样多个答案，再用 verifier 选最优。
Self-Consistency	Self-Consistency	自一致性	多条推理路径投票或聚合。
Verifier	Verifier	验证器	判断答案、步骤、代码或状态是否正确。
MCTS	Monte Carlo Tree Search	蒙特卡洛树搜索	通过搜索树探索候选推理路径。
x	Prompt	输入问题	用户问题或任务。
y	Response	模型回答	模型生成的完整回答或轨迹。
y_t	Reasoning Step	第 t 个推理步骤	推理轨迹中的一步。
$R(x,y)$	Reward	奖励	对回答或轨迹的得分。

本章目标

- 理解 reasoning model 和普通 instruction model 的区别。
- 能解释为什么 test-time compute 可以提升复杂任务表现。
- 掌握 PRM、ORM、verifier、best-of-n、self-consistency 的关系。
- 能讲清 DeepSeek-R1、OpenAI o-series、Qwen3 等报告里常见的 RLVR / thinking mode。
- 理解 reasoning 带来的安全和评测新问题。

Reasoning Model 是什么

普通 chat model 倾向于直接生成答案。Reasoning model 会在复杂任务上使用更多内部推理预算、采样或工具调用，再输出最终答案。

核心变化：

- 训练目标从“像高质量答案”变成“能通过验证和测试”。
- 推理阶段从单次生成变成可搜索、可验证、可反思。
- 对数学、代码、科学、工具调用、视觉推理更有效。

面试表达：

Reasoning model 的本质不是神秘的“会思考”，而是训练和推理范式变化：post-training 中用可验证奖励强化长推理轨迹，inference 时给更多 compute 让模型搜索、检查和修正。

Test-time Compute

Test-time compute 包括：

类型	English	中文	例子
Longer Thinking	Longer Thinking	更长推理	允许更多 hidden reasoning tokens。
Best-of-N	Best-of-N Sampling	多采样择优	采样多个答案，用 verifier 选最好。
Self-Consistency	Self-Consistency	自一致性	多条链投票。
Tool Execution	Tool Execution	工具执行	Python、browser、compiler、unit test。
Search	Search	搜索	tree search、beam、planner-executor。

关键直觉：

- 对不可验证任务，更多推理可能只是更长的幻觉。
- 对可验证任务，更多推理可以带来“生成-验证-修正”的收益。
- 现代 API 常提供 reasoning effort / thinking budget，本质就是控制质量、延迟、成本的旋钮。

PRM、ORM 与 Verifier

ORM 只看最终结果：

$$R_{ORM}(x,y)=I[final(y)=target]$$

PRM 看中间步骤：

$$R_{PRM}(x,y_{1:t}) \rightarrow \{0,1\} \text{ or } \mathbb{R}$$

Verifier 更泛化，可以检查：

- 数学最终答案。
- 代码单元测试。
- 工具调用最终数据库状态。
- RAG 引用是否支持答案。
- agent 是否完成用户目标。

面试回答：

ORM 信号简单但稀疏，PRM 信号密集但贵且可能奖励表面推理。Verifier 可以用于训练，也可以用于 inference-time reranking。现代 reasoning 系统常把它们组合起来。

RLVR 训练闭环

RLVR 典型流程：

1. 给模型一个问题 x 。
2. 模型采样多个回答或轨迹 $y_1, \dots, y_n$ 。
3. 外部 verifier 给 reward。
4. 用 RL 更新模型，使高 reward 轨迹概率升高。

可以写成：

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [R(x,y)]$$

为什么它有效：

- 数学和代码天然有 reward。
- agent 可以用最终状态做 reward。
- 模型能从失败样本中学习策略。
- 相比人工偏好，更可扩展。

为什么它危险：

- reward 写错会被 exploit。
- benchmark 泄漏会导致虚高。

- 模型可能学会 shortcut，而不是学会真实能力。
- 长 CoT 可能变成不可审计的内部策略。

GRPO 的直觉

GRPO 给同一个问题采样一组回答，用组内相对表现构造 advantage：

$$A_i = r_i - \bar{r} / \sigma_r$$

其中：

$$\bar{r} = 1 / G \sum_{j=1}^G r_j$$

优点：

- 不需要 value model。
- 显存更省。
- 很适合大模型上做 reasoning RL。

局限：

- 如果一组样本全对或全错，学习信号会弱。
- 对 reward 稀疏任务，采样策略很重要。
- 相对 reward 会鼓励组内胜出，不保证绝对质量。

DeepSeek-R1 的训练启发

DeepSeek-R1 把 reasoning RL 推到开源社区视野里。核心启发：

- R1-Zero 显示大规模 RL 可以让 reasoning behavior 自发出现。
- 纯 RL 可能出现可读性差、语言混杂等问题。
- R1 用 cold-start data + 多阶段 RL 改善可读性和稳定性。
- distilled models 说明大模型 reasoning 轨迹可以迁移到小模型。

面试表达：

DeepSeek-R1 的意义不只是模型效果，而是证明了“可验证 reward + 大规模 RL”可以显著激发推理能力。但它也暴露出 reasoning trace 可读性、语言混杂、reward 设计和安全监控问题。

Qwen3 的 Thinking / Non-thinking 统一

Qwen3 技术报告强调一个实用方向：同一个模型支持 thinking mode 和 non-thinking mode，并通过 thinking budget 控制推理深度。

面试可以这样讲：

这说明产业界不希望永远让用户手动切模型。更理想的系统是统一模型或路由器根据任务复杂度决定是否深度推理，在成本、延迟和质量之间动态折中。

CoT 监控与隐藏推理

Reasoning model 带来一个新问题：如果模型的真实推理过程不展示，怎么监控安全？

需要区分：

- 给用户看的解释。
- 模型内部 hidden CoT。
- 系统用于安全审计的 trace。

风险：

- 展示完整 CoT 可能泄露隐私、安全策略或训练数据。
- 不展示 CoT 会降低可解释性。
- 模型可能出现 reasoning-output discrepancy，即内部想法和外部回答不一致。

面试表达：

CoT 不是越透明越好。产品上常展示摘要而不是原始推理；安全上需要内部监控、红队、行为 eval 和工具轨迹审计一起做。

Reasoning Eval

常见评测：

Benchmark	English	中文	测什么
AIME	American Invitational Mathematics Examination	美国邀请数学竞赛	数学推理。
GPQA	Graduate-Level Google-Proof QA	研究生级问答	科学知识和推理。
FrontierMath	Frontier Math Benchmark	前沿数学基准	难数学问题。
SWE-bench	Software Engineering Benchmark	软件工程基准	真实代码 issue 修复。
Terminal-Bench	Terminal Benchmark	终端任务基准	命令行多步任务。
BrowseComp	Browsing Competition Benchmark	浏览检索基准	隐蔽信息查找。

Benchmark	English	中文	测什么
TAU-bench	Tool-Agent-User Benchmark	工具-智能体-用户基准	多轮业务工具任务。

不要只报 benchmark 分数。要追问：

- 是否用了 tool?
- 是否用了 high reasoning effort?
- 是否用了多次采样?
- 是否有外部 scaffold?
- 任务集是否污染?
- 结果是 pass@1 还是 best-of-n?

本章面试高频问题

1. Reasoning model 和普通 LLM 的区别?

普通 LLM 主要直接生成答案。Reasoning model 通过 RL 和 inference-time compute 学会更长的搜索、验证和修正，尤其适合数学、代码、科学、工具调用等可验证任务。

2. 为什么 PRM 有用?

PRM 给中间步骤提供监督，可以减少只看最终答案时的 credit assignment 问题。但 PRM 标注贵，也可能被模型学成表面推理模板。

3. 为什么 RLVR 比 RLHF 更适合代码和数学?

代码和数学有自动验证器，比如单元测试、编译器、答案检查器。相比人类偏好，这些 reward 更便宜、更客观、更可扩展。

4. 为什么更多 thinking 不一定更好?

如果任务不可验证，模型可能只是生成更长、更自信的错误推理。更多 thinking 还会增加延迟、成本和安全审计难度。

5. 面试被问 DeepSeek-R1 怎么答?

答训练范式：R1-Zero 用大规模 RL 激发推理；R1 加 cold-start data 和多阶段训练改善可读性；核心贡献是证明 verifiable reward + RL 可以扩展 reasoning，但也带来 reward hacking、语言混杂、CoT 监控等问题。

本章 References

- [Let's Verify Step by Step](#)
- [DeepSeekMath: Pushing the Limits of Mathematical Reasoning](#)
- [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)
- [Qwen3 Technical Report](#)
- [OpenAI o3 and o4-mini System Card](#)
- [Deliberative Alignment: Reasoning Enables Safer Language Models](#)
- [Think Fast: Estimating No-CoT Task-Completion Time Horizons of Frontier AI Models](#)

第 10 章：Agentic Systems、Coding Agent 与长期任务

本章符号说明

符号/缩写	English	中文	含义
Agentic AI	Agentic Artificial Intelligence	智能体式 AI	能持续规划、执行、观察和修正的 AI 系统。
Coding Agent	Coding Agent	编程智能体	能读仓库、改代码、跑测试、修 bug 的 agent。
Computer Use	Computer Use	计算机使用	模型操作 GUI、浏览器、文件和桌面应用的能力。
GUI	Graphical User Interface	图形用户界面	agent 可以点击、输入、观察的可视化界面。
Browser Agent	Browser Agent	浏览器智能体	能搜索、点击、阅读网页并完成网页任务的 agent。
Long-horizon Task	Long-horizon Task	长周期任务	需要很多步骤、较长时间和状态保持的任务。
API	Application Programming Interface	应用程序编程接口	agent 调用外部业务系统或工具的接口。
Tool Use	Tool Use	工具调用	使用 shell、browser、editor、database、API 等工具。
Tool Error	Tool Error	工具错误	工具调用参数错、执行失败、状态不一致等。
Agent Harness	Agent Harness	智能体执行框架	管理上下文、工具、权限、状态、trace、恢复和验证的系统层。
Scaffold	Scaffold	外部脚手架	agent 外层的控制循环、工具、记忆和验证代码。
Planner	Planner	规划器	拆任务和安排顺序的模块。
Executor	Executor	执行器	执行工具和产生修改的模块。
Critic	Critic	评审器	审查计划、代码、答案和安全风险的模块。
Verifier	Verifier	验证器	运行测试、检查状态或引用的模块。

符号/缩写	English	中文	含义
SWE-bench	Software Engineering Benchmark	软件工程基准	真实 GitHub issue 修复任务。
Terminal-Bench	Terminal Benchmark	终端任务基准	命令行环境长期任务。
OSWorld	Operating System World Benchmark	操作系统世界基准	GUI/computer-use 任务评测。
TAU-bench	Tool-Agent-User Benchmark	工具-智能体-用户交互基准	业务 API 多轮交互任务。
BrowseComp	Browsing Competition Benchmark	浏览检索基准	难查信息的网页浏览任务。

本章目标

- 理解 agentic system 的核心结构。
- 能讲清 coding agent 和普通代码补全的差别。
- 能设计一个生产可用 agent 的状态、工具、权限、验证和观测方案。
- 能识别 agent eval 的常见陷阱。

Agentic AI 的核心循环

Agent 的最小闭环：

```
goal -> plan -> act -> observe -> verify -> update plan -> finish
```

和普通 LLM API 的差异：

- 普通 LLM 主要是单轮生成。
- Agent 是多轮决策系统。
- Agent 的能力来自模型 + scaffold + tools + verifier。
- Agent 的风险来自写权限、长上下文、外部不可信信息和自动执行。

面试表达：

Agentic AI 不是单个模型能力，而是一个闭环系统。模型负责推理和决策，scaffold 负责状态、工具、预算、权限和验证。生产系统真正难的是可控性和可评估性。

Coding Agent

Coding agent 的典型流程：

1. 读 issue / 用户需求。

- 2. 搜索仓库结构。
- 3. 找相关文件和测试。
- 4. 制定修改计划。
- 5. 编辑代码。
- 6. 运行测试或静态检查。
- 7. 根据失败日志修复。
- 8. 输出 diff、验证结果和风险。

和代码补全的区别：

能力	Code Completion	Coding Agent
上下文	当前文件附近	整个仓库、issue、测试、历史日志
动作	生成片段	搜索、编辑、运行、调试、验证
反馈	人类看结果	工具和测试实时反馈
任务长度	秒级	分钟到小时级
评价	代码片段质量	最终 issue 是否解决

Long-horizon 为什么难

长期任务常见失败模式：

- 目标漂移：做着做着偏离原始需求。
- 状态遗忘：忘了已经尝试过什么。
- 局部修复：修了当前测试，破坏其他行为。
- 工具错误：命令失败但模型没发现。
- 循环卡住：重复同一动作。
- 过度修改：改太多无关文件。
- 自信汇报：没有验证却声称完成。

解决思路：

- 明确任务状态机。
- 每步记录 trace。
- 对写操作做 diff 和最小权限。
- 测试失败必须回读日志。
- 设置预算、重试上限和终止条件。
- 最终输出必须包含验证证据。

工具和权限设计

Agent 工具不是越多越好。每个工具都要明确：

设计项	English	中文	说明
Schema	Tool Schema	工具接口	参数类型、必填项、约束。
Permission	Permission	权限	read/write/delete/network/payment 等。
Sandbox	Sandbox	沙箱	限制文件系统、网络 and 命令。
Approval	Approval Gate	审批门	高风险动作需要人类确认。
Timeout	Timeout	超时	防止无限执行。
Rollback	Rollback	回滚	出错可恢复。
Audit Log	Audit Log	审计日志	记录动作和结果。

面试回答：

我会把工具按风险分层：只读工具默认允许，写工具需要 diff，破坏性工具需要审批，外部副作用工具需要强约束和审计。

多 Agent 与 Agent Swarm

多 agent 不等于更强，常见模式包括：

- Planner + Executor：一个规划，一个执行。
- Developer + Reviewer：一个写，一个审。
- Researcher + Synthesizer：一个查资料，一个汇总。
- Parallel Attempts：多个 agent 并行尝试，再由 verifier 选择。
- Specialist Agents：不同 agent 负责前端、后端、测试、文档。

风险：

- 通信成本高。
- 错误会互相放大。
- 没有统一状态时容易冲突。
- verifier 不强时，多 agent 只是更贵的幻觉。

面试表达：

多 agent 的价值在于并行探索和角色分工，但必须有共享状态、冲突解决和最终 verifier。否则只是把单 agent 的不稳定放大。

Agentic Coding 评测

常见 benchmark：

Benchmark	English	中文	关注点
SWE-bench	Software Engineering Benchmark	软件工程基准	真实 GitHub issue 修复。
SWE-bench Verified	SWE-bench Verified	人工验证软件工程基准	更可靠的真实 issue 子集。
Terminal-Bench	Terminal Benchmark	终端任务基准	命令行多步任务。
OSWorld	Operating System World	操作系统世界	GUI/computer use。
BrowseComp	Browsing Competition Benchmark	浏览检索基准	多步网页查找。
TAU-bench	Tool-Agent-User Benchmark	工具-用户交互基准	业务 API、用户对话和规则遵循。

评测时必须问：

- 是否用了同样 scaffold?
- 是否允许浏览器、shell、测试?
- 是否 pass@1 还是多次采样?
- 是否有隐藏测试?
- 是否计算成本和延迟?
- 是否记录 tool error?
- 是否做安全和权限评估?

Agent Harness 为什么重要

同一个 base model 放在不同 agent harness 里，表现可能完全不同。Harness 会决定：

- 模型看到多少上下文。
- 是否有文件搜索、浏览器、shell、测试运行器。
- 工具失败后是否自动恢复。
- 是否允许并行尝试。
- 是否有 reviewer / verifier。
- 写操作是否需要审批。
- 最终 trace 是否可回放。

面试表达：

Agent 能力不是 base model 单独决定的，而是 model + scaffold + harness + tools + budget 的组合。2026 年看 coding agent benchmark 时，必须问清楚 harness 和 protocol，否则分数不可比。

Agent 安全

Agent 比聊天模型风险更高，因为它可以行动。

关键风险：

- Prompt injection：网页或文档指挥模型忽略系统指令。
- Data exfiltration：外部内容诱导模型泄露机密。
- Tool misuse：模型调用不该调用的工具。
- Over-delegation：用户把高风险决策完全交给 agent。
- Reward hacking：为了通过测试修改测试或绕过规则。

安全策略：

1. 工具输出默认不可信。
2. 明确系统指令优先级。
3. 高风险动作 human-in-the-loop。
4. 审计每个 tool call。
5. verifier 检查最终状态，不只检查回答文本。
6. 对 prompt injection 做专门红队数据集。

生产系统 Observability

Agent 上线要监控：

- task success rate。
- tool call count。
- tool error rate。
- retry count。
- latency。
- token cost。
- human escalation rate。
- rollback rate。
- safety violation rate。
- 用户修正率和满意度。

面试表达：

Agent 产品不是 demo 成功就行。要能观察每一步 tool call、失败原因、成本、用户接管点和安全事件，否则出了问题无法 debug。

本章面试高频问题

1. Coding agent 为什么比代码补全难？

因为它要理解仓库、定位 bug、编辑多个文件、运行测试、读失败日志、持续修正，并保证最终状态正确。这是长周期闭环任务，不是单次 token completion。

2. 为什么 benchmark 结果经常不可比？

不同模型可能用了不同 scaffold、工具、采样次数、预算、测试可见性和人工干预。agent benchmark 必须报告 evaluation protocol。

3. Agent 怎么防 prompt injection?

把工具输出作为不可信数据，区分系统指令/用户指令/外部内容；敏感工具做权限隔离和审批；最终动作由 verifier 和 policy guard 检查。

4. 多 agent 有什么价值?

适合并行探索、角色分工和互审。但只有在有共享状态、冲突解决、强 verifier 时才稳定，否则会增加成本和错误传播。

5. 设计一个 coding agent 项目怎么讲?

按“任务-工具-状态-验证-评测-安全”讲。任务是 issue 修复；工具是代码搜索、编辑、shell、测试；状态是计划和尝试记录；验证是单测/回归测试；评测是 SWE-bench 类任务和内部 issue；安全是最小权限、diff 审批和日志审计。

本章 References

- [ReAct: Synergizing Reasoning and Acting in Language Models](#)
- [Toolformer: Language Models Can Teach Themselves to Use Tools](#)
- [AgentBench: Evaluating LLMs as Agents](#)
- [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)
- [TAU-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains](#)
- [BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents](#)
- [Kimi K2: Open Agentic Intelligence](#)
- [Open-World Evaluations for Measuring Frontier AI Capabilities](#)
- [Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows](#)
- [Claude Opus 4.7](#)
- [Introducing GPT-5.5](#)

第 11 章：2026 SOTA 技术报告阅读地图

本章符号说明

符号/缩写	English	中文	含义
SOTA	State of the Art	当前最优	某任务或评测上的前沿水平。
API	Application Programming Interface	应用程序编程接口	模型服务和工具调用的接口。
GPT	Generative Pre-trained Transformer	生成式预训练 Transformer	OpenAI GPT 系列模型名称中的缩写。
GLM	General Language Model	通用语言模型	智谱 GLM 系列模型名称中的缩写。
K2	Kimi K2	Kimi K2 模型	Moonshot AI 的 agentic MoE 模型系列。
R1	DeepSeek-R1	DeepSeek-R1 模型	DeepSeek 的 reasoning model 系列。
System Card	System Card	系统卡	描述模型能力、限制、安全评估和部署策略的官方报告。
Tech Report	Technical Report	技术报告	描述模型架构、训练、数据、评测和安全的报告。
Frontier Model	Frontier Model	前沿模型	当前能力接近行业最前沿的大模型。
Reasoning Model	Reasoning Model	推理模型	会分配额外推理计算解决复杂任务的模型。
Hybrid Reasoning	Hybrid Reasoning	混合推理	同一模型或系统支持快速回答和深度推理。
Router	Router	路由器	根据任务复杂度选择模型或推理模式的组件。
MoE	Mixture of Experts	混合专家模型	每个 token 只激活部分专家参数。
RLVR	Reinforcement Learning with Verifiable Rewards	可验证奖励强化学习	用自动验证 reward 优化推理、代码、工具任务。
GRPO	Group Relative Policy Optimization	组相对策略优化	DeepSeek-R1 相关报告中常见的 RL 算法。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward model 并优化策略。

符号/缩写	English	中文	含义
SFT	Supervised Fine-Tuning	监督微调	用高质量指令数据微调模型。
ARC	Agentic, Reasoning, Coding	智能体、推理、编程	GLM 系列报告中强调的三类能力。
DSA	DeepSeek Sparse Attention	DeepSeek 稀疏注意力	GLM-5 报告采用的长上下文高效注意力路线。
IndexShare	Index Sharing Sparse Attention	索引共享稀疏注意力	GLM-5.2 repo 中提出的 1M context 稀疏注意力优化。
MTP	Multi-Token Prediction	多 token 预测	GLM-5.2 repo 中用于提升 speculative decoding 接受长度的机制。
RLCS	Reinforcement Learning with Curriculum Sampling	课程采样强化学习	GLM-4.5V 报告中用于视觉推理的 RL 方法。
Agentic Coding	Agentic Coding	智能体式编程	模型能规划、修改、测试、调试代码任务。
GUI	Graphical User Interface	图形用户界面	多模态 agent 操作的网页或桌面界面。
UI	User Interface	用户界面	文档、网页、应用界面等视觉任务对象。
SWE	Software Engineering	软件工程	coding agent 和真实仓库任务所在领域。
STEM	Science, Technology, Engineering, Mathematics	科学、技术、工程、数学	高难推理和多模态评测常见领域。
Terminal-Bench	Terminal Benchmark	终端任务基准	衡量模型在 shell/terminal 中完成长期任务的 benchmark。
SWE-bench Pro	Software Engineering Benchmark Pro	软件工程进阶基准	更高难度的软件工程 agent benchmark。
Long Context	Long Context	长上下文	支持数十万到百万 token 级输入。
Multimodal	Multimodal	多模态	支持文本、图像、音频、视频等输入或输出。

本章目标

- 知道截至 2026-06-21，大模型技术报告的主线是什么。
- 能把 OpenAI、Anthropic、Google、DeepSeek、Qwen、Kimi、GLM 的报告按技术维度比较。

- 面试中能从“训练方法、推理方式、agentic 能力、评测协议、安全策略”五个角度讲模型。
- 不背模型名，而是读出趋势。

读 SOTA 报告的框架

看任何模型报告，按这 8 个问题读：

1. 模型是 dense 还是 MoE?
2. 是单模型，还是 router/system-of-models?
3. 是否支持 thinking / reasoning effort?
4. post-training 里用了 SFT、preference、RLHF、RLVR、tool RL 里的哪些?
5. agentic 能力如何评估？是否给了 scaffold、工具、预算？
6. 长上下文和多模态能力怎么做？是真支持还是只在特定 preview?
7. 安全评测覆盖哪些高风险域？是否有 system card?
8. benchmark 是否可比？是否用了 high compute、best-of-n、tool access?

OpenAI: o-series、GPT-5 与 GPT-5.5

OpenAI o3 / o4-mini system card 的重点：

- reasoning model 结合完整工具能力。
- 模型可以在推理过程中使用 web、Python、image/file analysis 等工具。
- o-series 使用 large-scale RL on chains of thought。
- deliberative alignment 用安全规范推理提升安全边界遵循。

GPT-5 system card 的重点：

- GPT-5 是 unified system，而不只是单一模型。
- 系统包含 fast model、deeper reasoning model 和 real-time router。
- 强调减少 hallucination、改善 instruction following、降低 sycophancy。
- 引入 safe-completions：在安全范围内尽量有帮助，而不是简单拒答。

GPT-5.5 官方 release 的重点：

- 面向 agentic coding、Terminal-Bench、SWE-Bench Pro、BrowseComp 等长任务。
- 强调更少 token、更少 retry、更高质量输出。
- 在 Codex 和 ChatGPT 中用于复杂工程、研究和电脑工作流。
- API 部署需要额外 safeguards，说明 frontier model 上线已经和安全治理强绑定。

面试表达：

OpenAI 这条线的关键词是 unified system、router、reasoning effort、tool-augmented reasoning 和 safe-completions。它把“模型能力”和“产品系统”绑定得更紧：同一个用户请求可能被路由到快模型、深度推理模型或工具增强流程。

Anthropic: Claude 4 到 Opus 4.7

Claude 4 报告重点：

- extended thinking with tool use。
- parallel tool execution。
- memory improvements。
- coding 和 agentic tasks 上减少 shortcut / loophole 行为。

Claude Opus 4.7 官方 release 的重点：

- 复杂长任务和高级软件工程能力提升。
- 更重视 self-verification：完成前自己检查输出。
- 图像分辨率和专业文档/UI/slide 任务增强。
- 对高风险 cybersecurity 能力做差异化削弱和自动防护。

面试表达：

Anthropic 这条线的关键词是 long-running task、tool use、memory、self-verification 和 safety gating。它特别强调模型作为“长期协作者”而不是单轮问答模型。

Google DeepMind： Gemini 2.5

Gemini 2.5 technical report 的重点：

- Gemini 2.5 Pro / Flash 是 thinking model 或 hybrid reasoning model。
- 原生多模态，文本、图像、音频、视频和长上下文结合。
- 支持 1M token 级长上下文，Gemini 2.5 Pro 可处理长视频内容。
- Sparse MoE 架构，提高容量/成本比。
- Flash 走 controllable thinking budget，覆盖质量、成本、延迟 Pareto frontier。

面试表达：

Gemini 2.5 的关键词是 native multimodality、long context、MoE 和 controllable thinking budget。它的差异化在于把长上下文、多模态和 agentic workflow 结合，而不是只做纯文本推理。

DeepSeek-R1： 开源 reasoning RL 里程碑

DeepSeek-R1 技术报告重点：

- R1-Zero 用大规模 RL，不经过传统 SFT cold start，也能出现推理能力。
- R1 引入 cold-start data 和多阶段训练，解决可读性和语言混杂问题。
- GRPO / verifiable reward 成为开源 reasoning model 训练的重要范式。
- 蒸馏出多个小模型，说明 reasoning 能力可以通过轨迹迁移。

面试表达：

DeepSeek-R1 的影响在于训练范式：它让社区看到大规模 RLVR 可以激发 reasoning behavior，并推动 GRPO、verifier、distillation 成为开源模型训练标配。

Qwen3: Thinking / Non-thinking 统一

Qwen3 technical report 重点：

- 同一模型支持 thinking mode 和 non-thinking mode。
- 引入 thinking budget，在性能、延迟和成本之间调节。
- 同时覆盖 dense 和 MoE，模型规模从小模型到 235B。
- 多语言从 Qwen2.5 的 29 种扩到 119 种语言/方言。
- 强调 agent tasks、code、math 和 multilingual 能力。

面试表达：

Qwen3 的关键不是单个分数，而是统一 reasoning 和 non-reasoning 的产品形态。它说明开源模型也在走“一个模型动态调 thinking budget”的方向。

Kimi K2 / K2.5: Open Agentic Intelligence

Kimi K2 技术报告重点：

- 1T total parameters、32B activated parameters 的 MoE。
- MuonClip optimizer 缓解训练不稳定。
- 15.5T tokens 预训练。
- post-training 里有大规模 agentic data synthesis 和 joint RL。
- 强项集中在 agentic tasks、software engineering 和 coding。

Kimi K2.5 技术报告重点：

- 多模态 agentic model。
- joint text-vision pretraining、zero-vision SFT、joint text-vision RL。
- Agent Swarm：并行分解复杂任务，降低延迟。

面试表达：

Kimi 这条线的关键词是 open agentic intelligence：不只追求聊天能力，而是把软件工程、工具交互、多模态和 agent swarm 放进训练目标。

GLM-5 / GLM-5.1 / GLM-5.2: Agentic Engineering

GLM-5 技术报告重点：

- 从 vibe coding 走向 agentic engineering。
- 延续 ARC：Agentic、Reasoning、Coding 三类能力一起优化。

- GLM-5 规模提升到 744B total parameters、40B activated parameters。
- 采用 DSA，降低长上下文训练和推理成本。
- 引入异步 reinforcement learning infrastructure，把 generation 和 training 解耦，提高 post-training 效率。
- 强调 complex systems engineering、long-horizon agentic tasks 和真实代码工程任务。

GLM-5.1 官方 repo 重点：

- 面向 agentic engineering。
- 强调模型能在更长 session 中持续实验、读结果、识别 blocker、迭代策略。
- 关注 SWE-Bench Pro、NL2Repo、Terminal-Bench 2.0 等工程基准。

GLM-5.2 官方 repo 重点：

- 最新 flagship model，主打 long-horizon tasks。
- 支持 solid 1M-token context，面向长期工作流。
- coding 能力支持 multiple thinking effort levels，在效果和延迟之间折中。
- 提出 IndexShare，在 1M context 下复用稀疏注意力 index，降低 per-token FLOPs。
- 改进 MTP layer，用于 speculative decoding，提升 acceptance length。
- 官方 repo 报告了 Terminal-Bench 2.1 和 SWE-bench Pro 等 coding benchmark 改进。

GLM-4.5V / GLM-4.1V-Thinking 仍然值得了解：

- 多模态 reasoning。
- RLCS 用于解锁视觉推理能力。
- 覆盖 STEM、video understanding、content recognition、coding、grounding、GUI agents、long document。

面试表达：

GLM 这条线已经从 GLM-4.5 的 ARC 走到 GLM-5 系列的 agentic engineering。重点不是只看开源模型分数，而是看 1M context、异步 RL、long-horizon coding、thinking effort、Terminal-Bench / SWE-bench Pro 这类长期工程任务怎么一起推动。

横向对比表

系列	English	中文	关键词	面试重点
OpenAI GPT-5/5.5	OpenAI GPT-5/5.5	OpenAI GPT-5/5.5	router、safe-completions、agentic coding	unified system，不是单模型。
OpenAI o-series	OpenAI o-series	OpenAI o 系列	CoT RL、tool reasoning、deliberative alignment	reasoning + tool use + safety reasoning。
Claude Opus 4.7	Claude Opus 4.7	Claude Opus 4.7	long-running tasks、self-verification、safety gating	长任务稳定性和工程协作。

系列	English	中文	关键词	面试重点
Gemini 2.5	Gemini 2.5	Gemini 2.5	native multimodal、1M context、MoE、thinking budget	长上下文 + 多模态 + agentic workflow。
DeepSeek-R1	DeepSeek-R1	DeepSeek-R1	RLVR、GRPO、distillation	开源 reasoning RL 范式。
Qwen3	Qwen3	Qwen3	thinking/non-thinking unified、MoE、multilingual	统一 thinking budget 和开源生态。
Kimi K2/K2.5	Kimi K2/K2.5	Kimi K2/K2.5	agentic data synthesis、joint RL、Agent Swarm	agentic intelligence 和并行任务分解。
GLM-5.2/5.1/5	GLM-5.2/5.1/5	GLM-5 系列	1M context、IndexShare、MTP、async RL、agentic engineering	long-horizon coding 和工程任务。
GLM-4.5V	GLM-4.5V	GLM-4.5V	RLCS、multimodal reasoning、GUI agents	多模态 RL 和视觉推理。

2026 技术趋势总结

1. 从单模型到系统模型

前沿产品越来越像 system-of-models: router、fast model、thinking model、tools、memory、verifier 一起工作。

2. 从 SFT 到 RLVR

数学、代码、工具调用、浏览、GUI 都有可验证 reward，RLVR 成为提升 reasoning 和 agentic 能力的核心。

3. 从聊天到长任务

SOTA 模型的竞争点从“答题准确率”转向“能不能稳定完成长周期工作”。

4. 从单 agent 到多 agent / parallel attempts

复杂任务常用并行尝试、候选筛选、agent swarm，但最终仍依赖 verifier。

5. 从安全拒答到安全完成

安全系统从二元拒答转向 safe-completions、deliberative alignment、cyber gating、preparedness framework。

6. 从 VLM 到多模态 agent

多模态不只是看图回答，而是视频、GUI、文档、浏览器、代码和工具环境的统一推理。

本章面试高频问题

1. 2026 年大模型最重要的趋势是什么？

从“更大预训练模型”转向“reasoning + tools + verifiable RL + long-horizon agents”。模型能力、系统架构和安全部署越来越不可分。

2. OpenAI、Anthropic、Gemini 的差异怎么讲？

OpenAI 强调 unified system、router、agentic coding 和 safe-completions；Anthropic 强调 long-running task、tool use、self-verification 和安全 gating；Gemini 强调 native multimodality、long context、MoE 和 controllable thinking budget。

3. 开源模型最近为什么进步快？

DeepSeek-R1、Qwen3、Kimi K2、GLM-5.2 都把 reasoning RL、MoE、agentic data、distillation、thinking budget、long-horizon coding 等技术公开化，让社区可以复现和组合。

4. 技术报告里的 benchmark 怎么看？

先看是否用了 tool、high reasoning effort、多次采样、外部 scaffold 和隐藏测试。不同协议下的分数不能直接横比。

5. 面试如何展示“我跟进了 SOTA”？

不要背模型排行榜。讲趋势和机制：RLVR 为什么重要，agent eval 为什么难，long-context 何时替代不了 RAG，safe-completions 为什么比拒答更好，多模态 agent 为什么是下一阶段。

本章 References

- [OpenAI o3 and o4-mini System Card](#)
- [Introducing OpenAI o3 and o4-mini](#)
- [OpenAI GPT-5 System Card](#)
- [Introducing GPT-5.5](#)
- [Claude 4](#)
- [Claude Opus 4.7](#)
- [Gemini 2.5 Technical Report](#)
- [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)
- [Qwen3 Technical Report](#)
- [Kimi K2: Open Agentic Intelligence](#)
- [Kimi K2.5: Visual Agentic Intelligence](#)
- [GLM-5: from Vibe Coding to Agentic Engineering](#)
- [Z.ai GLM-5.2 / GLM-5.1 / GLM-5 official repo](#)
- [GLM-4.5: Agentic, Reasoning, and Coding Foundation Models](#)
- [GLM-4.1V-Thinking and GLM-4.5V](#)

第 12 章：RAG 系统、Agentic RAG 与评测

本章符号说明

符号/缩写	English	中文	含义
RAG	Retrieval-Augmented Generation	检索增强生成	从外部知识源检索证据，再让模型基于证据生成答案。
LLM	Large Language Model	大语言模型	基于大规模文本预训练的生成式语言模型。
IR	Information Retrieval	信息检索	从文档集合中找到相关信息的技术领域。
API	Application Programming Interface	应用程序编程接口	RAG 从业务系统实时取数时常用的接口。
DB	Database	数据库	存储结构化或半结构化数据的系统。
SQL	Structured Query Language	结构化查询语言	查询关系型数据库的语言。
PDF	Portable Document Format	可移植文档格式	企业文档和报告常见格式。
HTML	HyperText Markup Language	超文本标记语言	网页文档常见格式。
OCR	Optical Character Recognition	光学字符识别	从扫描件或图片中识别文字。
FAQ	Frequently Asked Questions	常见问题	企业知识库常见文档类型。
HR	Human Resources	人力资源	权限敏感文档常见来源之一。
BM25	Best Matching 25	BM25 排序函数	经典 sparse retrieval 关键词匹配方法。
Dense Retrieval	Dense Retrieval	稠密检索	用 embedding 向量相似度检索语义相关文档。
Sparse Retrieval	Sparse Retrieval	稀疏检索	用词项、倒排索引、BM25 等方法检索。
Hybrid Search	Hybrid Search	混合检索	组合 sparse 和 dense retrieval。
Embedding	Embedding	向量表示	文本、图片或多模态对象的稠密向量。

符号/缩写	English	中文	含义
Vector DB	Vector Database	向量数据库	存储 embedding 并支持近邻搜索的数据库。
Chunking	Chunking	文档切块	把文档切成可检索片段。
Query Rewrite	Query Rewrite	查询改写	将用户问题改写成更适合检索的查询。
Multi-query	Multi-query Retrieval	多查询检索	为同一问题生成多个检索查询，提高召回。
HyDE	Hypothetical Document Embeddings	假设文档向量	先生成假设答案/文档，再用它做向量检索。
Reranker	Reranker	重排序模型	对初召回结果重新排序，提高 top-k 证据质量。
Cross-encoder	Cross-encoder Reranker	交叉编码重排序器	同时编码 query 和 document 计算相关性。
GraphRAG	Graph Retrieval-Augmented Generation	图检索增强生成	用知识图谱、实体关系和社区摘要增强检索。
Agentic RAG	Agentic Retrieval-Augmented Generation	智能体式 RAG	让 agent 动态规划、检索、验证和补充证据。
Long-context RAG	Long-context Retrieval-Augmented Generation	长上下文 RAG	检索后把更大证据包交给长上下文模型。
ACL	Access Control List	访问控制列表	决定用户可访问哪些文档或字段的权限配置。
PII	Personally Identifiable Information	个人可识别信息	需要脱敏和权限保护的敏感信息。
Groundedness	Groundedness	有根据性	答案是否被给定证据支持。
Faithfulness	Faithfulness	忠实性	生成内容是否忠实于检索证据。
Citation Precision	Citation Precision	引用精确率	引用是否真正支持对应答案句。
Recall@k	Recall at k	前 k 召回率	gold evidence 是否出现在前 k 个结果中。
MRR	Mean Reciprocal Rank	平均倒数排名	第一个相关结果排名的倒数均值。
nDCG	Normalized Discounted Cumulative Gain	归一化折损累计增益	考虑排序位置和相关性等级的检索指标。
RAGAS	RAG Assessment	RAG 评估工具	常用于 RAG answer faithfulness、context relevance 等自动评测。

符号/缩写	English	中文	含义
REALM	Retrieval-Augmented Language Model Pre-Training	检索增强语言模型预训练	早期把检索接入语言模型预训练的代表工作。
q	Query	查询	用户问题或检索查询。
D	Document Corpus	文档库	可检索的文档集合。
d_i	Document Chunk	第 i 个文档块	文档切块后的检索单元。
$S(q, d_i)$	Retrieval Score	检索分数	query 与文档块的相关性分数。
C_k	Retrieved Context	前 k 个上下文	检索后交给模型的证据集合。

本章目标

- 能把 RAG 讲成一个完整系统，而不是一句“向量库 + prompt”。
- 能区分 retrieval error、rerank error、context packing error 和 generation error。
- 能解释 GraphRAG、Agentic RAG、long-context RAG 的使用场景。
- 能设计 RAG 评测集、指标、线上监控和权限边界。
- 能回答“长上下文模型是不是替代 RAG”这类面试追问。

为什么 2026 年仍然要会 RAG

长上下文模型越来越强，但 RAG 没有消失。原因很简单：

- 企业知识经常更新，不能每次重新训练模型。
- 私有文档需要权限隔离，而不是把所有内容塞进 prompt。
- 生产系统需要 citation、audit log、可回放和版本控制。
- 很多问题需要从结构化系统、数据库、网页、代码仓库实时读取。
- 长上下文更贵，且模型不一定能在长证据里稳定找到关键事实。

面试表达：

Long context 解决的是“能不能塞进去”，RAG 解决的是“从哪里找、谁有权限看、证据是否支持、结果能否审计”。两者不是替代关系，生产里常见组合是 retrieval + rerank + long-context packing + citation verification。

RAG Pipeline

一个生产级 RAG 通常分为两条链路。

离线索引链路：

1. 文档接入：PDF、HTML、Markdown、数据库、工单、代码仓库、表格。
2. 解析清洗：去噪、版面恢复、表格结构、图片 OCR。

3. 权限绑定：把 document、chunk、metadata 和 ACL 绑定。
4. Chunking：按标题、语义段、表格、代码符号或页面结构切块。
5. Embedding：生成 dense vector。
6. Sparse index：建立倒排索引或 BM25。
7. Metadata index：保留时间、作者、部门、版本、权限、来源。

在线查询链路：

1. Query understanding：识别意图、语言、实体、时间范围。
2. Query rewrite / decomposition：改写或拆成子问题。
3. Hybrid retrieval：BM25 + dense retrieval。
4. Rerank：cross-encoder 或 LLM rerank。
5. Context packing：去重、压缩、保留结构、控制 token budget。
6. Generation：基于证据回答。
7. Citation verification：检查答案句和引用是否一致。
8. Refusal / abstention：证据不足时拒答或要求澄清。

抽象公式：

$$C_k = \text{Top}K_{d_i \in D} S(q, d_i)$$

$$\hat{y} = \text{LLM}(q, C_k)$$

这里 C_k 是检索上下文，不等于“越多越好”。上下文太多会增加成本，也可能把模型注意力从关键证据上稀释掉。

Chunking 怎么设计

Chunking 是 RAG 里最容易被低估的环节。

策略	English	中文	适合场景	风险
Fixed-size Chunking	Fixed-size Chunking	固定长度切块	快速 baseline	切断语义和表格。
Recursive Chunking	Recursive Chunking	递归切块	按标题、段落、句子逐级切	依赖解析质量。
Semantic Chunking	Semantic Chunking	语义切块	FAQ、制度文档、知识库	成本更高，边界不稳定。
Layout-aware Chunking	Layout-aware Chunking	版面感知切块	PDF、报告、票据、表格	需要 OCR/layout 模型。
Code-aware Chunking	Code-aware Chunking	代码感知切块	函数、类、调用关系	需要语法树或符号索引。

策略	English	中文	适合场景	风险
Small-to-big Retrieval	Small-to-big Retrieval	小块召回大块扩展	先精确召回，再补上下文	扩展窗口过大可能污染。

面试重点：

Chunk 太小会丢上下文，chunk 太大会降低召回精度。生产里常用小块建索引、大块给模型，配合 metadata、标题路径和邻接窗口。

Retrieval：不要只会向量库

RAG 初召回通常要混合多种信号：

方法	English	中文	优点	缺点
BM25	Best Matching 25	关键词检索	精确匹配术语、编号、错误码	同义词和语义泛化弱。
Dense Retrieval	Dense Retrieval	稠密检索	语义匹配强	专有名词、数字、代码符号可能弱。
Hybrid Search	Hybrid Search	混合检索	同时覆盖关键词和语义	分数融合需要调参。
Metadata Filter	Metadata Filtering	元数据过滤	时间、部门、权限、版本可控	metadata 质量要求高。
Graph Retrieval	Graph Retrieval	图检索	适合实体关系、多跳问题	构图和维护成本高。
SQL / API Retrieval	Structured Retrieval	结构化检索	精确、实时、可权限控制	需要 schema 和工具调用能力。

分数融合可以简单写成：

$$S(q,d)=\alpha S_{dense}(q,d)+(1-\alpha)S_{sparse}(q,d)$$

其中 α 是 dense / sparse 的权重。工程上不一定线性融合，也可以用 rank fusion、learning-to-rank 或 reranker。

Reranking 与 Context Packing

初召回要追求 recall，rerank 要追求 precision。常见做法：

- 初召回 top 50 到 top 200。
- reranker 压到 top 5 到 top 20。
- 按来源去重，避免同一段重复占满上下文。
- 保留标题路径、页码、表格 header、时间戳。

- 对长文档做 section summary，再把原文关键段落保留给模型。

Context packing 的目标不是塞满窗口，而是让模型看到：

1. 足够回答问题的证据。
2. 冲突证据和时间版本。
3. 每条证据的来源。
4. 不该看到的内容被权限过滤掉。

GraphRAG

GraphRAG 适合这些问题：

- 需要跨文档、多实体、多跳关系。
- 用户问的是主题、趋势、组织结构、事件链，而不是单点事实。
- 文档量大，单纯 top-k chunk 容易碎片化。

典型流程：

1. 从文档中抽取 entity、relation、claim。
2. 建图并做社区发现。
3. 生成社区摘要或实体摘要。
4. 查询时结合图邻域、社区摘要和原文证据。

局限：

- 构图成本高。
- 抽取错误会传播。
- 图摘要可能丢细节。
- 权限和增量更新更复杂。

面试表达：

GraphRAG 不是万能增强版 RAG，它更适合全局主题和多跳关系问题。对于精确政策条款、合同字段、错误码检索，普通 hybrid retrieval 可能更可靠。

Agentic RAG

Agentic RAG 让模型不再一次性检索，而是在循环中动态决定下一步：

```
question -> plan -> retrieve -> read -> identify gap -> retrieve again -> verify -> answer
```

它适合：

- 复杂研究问题。
- 多跳问答。

- 需要查多个系统的企业问答。
- 需要先澄清权限、时间范围或实体歧义的问题。

风险：

- 延迟和成本更高。
- 检索路径更难复现。
- 如果没有 trace 和 verifier，agent 会把错误证据越滚越大。
- prompt injection 可以通过外部文档污染下一步工具调用。

RAG Evaluation

RAG 评测要拆开来：

层级	English	中文	指标
Retrieval Eval	Retrieval Evaluation	检索评测	Recall@k、MRR、nDCG、hit rate。
Rerank Eval	Reranking Evaluation	重排序评测	top-k precision、pairwise preference。
Context Eval	Context Evaluation	上下文评测	context relevance、context coverage、token waste。
Generation Eval	Generation Evaluation	生成评测	faithfulness、answer correctness、format。
Citation Eval	Citation Evaluation	引用评测	citation precision、unsupported claim rate。
Permission Eval	Permission Evaluation	权限评测	越权检索率、PII 泄漏率。
System Eval	System Evaluation	系统评测	端到端成功率、延迟、成本、拒答准确率。

一个常见误区：只用答案正确率评估 RAG。这样看不到错误来自哪里。

更好的 eval set 至少包含：

- query。
- gold answer。
- gold evidence chunk。
- allowed document scope。
- expected citations。
- unanswerable questions。
- conflicting evidence cases。
- stale document cases。
- permission-denied cases。

RAGAS 类自动评测的价值和风险

RAGAS 类工具常评估：

- context relevance：检索上下文是否相关。
- faithfulness：答案是否被上下文支持。
- answer relevance：答案是否回答了问题。
- answer correctness：答案与标准答案是否一致。

价值：

- 快速回归。
- 适合大量样本趋势分析。
- 能把 retrieval 和 generation 问题拆开。

风险：

- LLM judge 会有偏差。
- judge 可能看漏证据。
- 没有权限、成本、延迟、线上分布就不完整。
- 自动分数不能替代人工 gold set。

面试表达：

RAG eval 不能只靠 LLM-as-a-judge。我会用人工 gold evidence 做 retrieval eval，用 citation verifier 做 grounding eval，用人工抽检校准 judge，并把权限、延迟、成本纳入 system eval。

常见失败模式

失败模式	English	中文	例子	解决方向
Missing Recall	Missing Recall	召回缺失	关键文档没进 top-k	query rewrite、hybrid search、chunk 重做。
Wrong Evidence	Wrong Evidence	错误证据	检测到旧版本政策	metadata filter、freshness、rerank。
Context Pollution	Context Pollution	上下文污染	无关段落塞满窗口	rerank、去重、context packing。
Lost in the Middle	Lost in the Middle	中间遗忘	长上下文中忽略关键段	证据排序、摘要、引用定位。
Citation Drift	Citation Drift	引用漂移	引用 A 但答案来自 B	citation verification。
Permission Leak	Permission Leak	权限泄漏	普通用户检测到 HR 文档	ACL filter、query-time auth。

失败模式	English	中文	例子	解决方向
Prompt Injection	Prompt Injection	提示注入	文档要求模型泄露系统提示	工具输出不可信、指令分层。
Over-answering	Over-answering	过度回答	无证据仍编答案	abstention、confidence、verifier。

面试高频问题

1. RAG 为什么不能只用向量库？

向量检索语义泛化强，但对编号、时间、实体、代码符号和精确术语不一定稳定。生产里通常要 hybrid search、metadata filter、rerank、权限控制和 citation verification。

2. 长上下文模型会替代 RAG 吗？

不会完全替代。长上下文适合一次性阅读大材料；RAG 适合动态知识、权限隔离、引用审计和成本控制。实际系统经常组合使用。

3. 怎么定位 RAG 答错原因？

先看 gold evidence 是否被召回，再看 rerank 排名，再看 context 是否被正确打包，最后看生成是否忠实。不要直接把错误归因给 LLM。

4. Agentic RAG 比普通 RAG 强在哪里？

它能动态拆问题、多轮检索、发现证据缺口并补查。代价是更慢、更贵、更难复现，所以需要 trace、预算和 verifier。

5. 企业 RAG 最重要的工程风险是什么？

权限和证据。用户不能检到无权限内容，答案必须能被引用支持。否则系统即使看起来聪明，也不适合上线。

本章 References

- [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)
- [REALM: Retrieval-Augmented Language Model Pre-Training](#)
- [Microsoft GraphRAG](#)
- [RAGAS](#)
- [Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG](#)
- [Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers](#)

第 13 章：Eval Harness、Benchmark 与可复现评测

本章符号说明

符号/缩写	English	中文	含义
Eval	Evaluation	评测	对模型或系统能力、风险和成本的测量。
LLM	Large Language Model	大语言模型	被评测的语言模型或系统核心。
LM	Language Model	语言模型	LM Evaluation Harness 名称中的缩写。
VLM	Vision-Language Model	视觉语言模型	同时处理视觉和语言输入的模式。
RAG	Retrieval-Augmented Generation	检索增强生成	需要单独设计 retrieval、generation、citation 评测的系统。
API	Application Programming Interface	应用程序编程接口	模型或工具服务的调用接口。
Harness	Harness	评测/执行框架	管理数据、prompt、模型调用、工具、环境、打分和记录的系统层。
Eval Harness	Evaluation Harness	评测框架	把 benchmark 跑成可复现实验的执行框架。
Agent Harness	Agent Harness	智能体执行框架	管理 agent 上下文、工具、状态、权限、trace 和恢复的系统层。
Benchmark	Benchmark	基准测试	固定任务集合和指标，用于比较模型或系统。
Leaderboard	Leaderboard	排行榜	汇总 benchmark 分数的公开排名。
Protocol	Evaluation Protocol	评测协议	规定 prompt、采样、工具、预算、指标和报告方式。
Model Adapter	Model Adapter	模型适配器	把不同模型/API 统一成 harness 可调用接口。
Task Adapter	Task Adapter	任务适配器	把数据样本转成 prompt、工具环境和期望输出。
Dataset Adapter	Dataset Adapter	数据集适配器	读取、过滤、切分和版本化数据集。

符号/缩写	English	中文	含义
Prompt Template	Prompt Template	提示模板	评测时固定的输入格式。
Runner	Runner	运行器	批量执行模型调用或 agent 任务的组件。
Sandbox	Sandbox	沙箱	隔离文件系统、网络 and 外部副作用的环境。
Environment	Environment	环境	agent 可以观察和行动的任务世界。
Oracle	Oracle	标准判定器	能可靠判断答案或最终状态是否正确的程序/标注。
Scorer	Scorer	打分器	将输出转成数值分数的组件。
Judge	Judge	裁判模型/评审者	用模型或人类判断开放式输出质量。
LLM-as-a-Judge	LLM as a Judge	大模型裁判	用 LLM 给答案、引用或轨迹评分。
HELM	Holistic Evaluation of Language Models	语言模型整体评测	Stanford CRFM 的综合评测框架。
VHELM	Vision Holistic Evaluation of Language Models	视觉语言模型整体评测	HELM 思路扩展到 VLM 的评测。
MMLU	Massive Multitask Language Understanding	大规模多任务语言理解基准	常见知识和推理评测。
MRR	Mean Reciprocal Rank	平均倒数排名	检索排序指标。
F1	F1 Score	F1 分数	precision 和 recall 的调和平均。
SWE-bench	Software Engineering Benchmark	软件工程基准	真实 GitHub issue 修复任务。
TAU-bench	Tool-Agent-User Benchmark	工具-智能体-用户交互基准	业务工具调用和用户交互任务。
Trace	Trace	轨迹日志	记录 prompt、tool call、observation、输出和分数。
Trajectory	Trajectory	执行轨迹	agent 从开始到结束的动作序列。
Replay	Replay	回放	用保存的输入、状态和工具结果复现实验。

符号/缩写	English	中文	含义
pass@k	Pass at k	k 次通过率	采样 k 次至少一次成功的概率指标。
Contamination	Data Contamination	数据污染	测试集进入训练数据或 prompt 导致分数虚高。
Decontamination	Decontamination	去污染	检查并移除训练/评测重叠的过程。
CI	Continuous Integration	持续集成	代码变更后自动运行测试和评测的工程流程。
M	Model	模型	被评测的基础模型或系统。
T	Task Set	任务集合	benchmark 中的任务样本集合。
H	Harness	评测框架	执行评测的系统层。
s_i	Score	第 i 个样本分数	单个任务的评分。

本章目标

- 理解 benchmark、eval、harness、leaderboard 的区别。
- 能解释为什么 2026 年要报告 model + harness + scaffold，而不是只报 model 名。
- 能设计 LLM、RAG、coding agent、VLM 的评测 harness。
- 能识别 benchmark 污染、工具预算不公平、LLM judge 偏差、pass@k 滥用。
- 能把 eval 接入研发闭环，而不是只跑一次排行榜。

Harness 是什么

Harness 本质上是“把评测跑起来并保证可复现”的系统层。

它通常负责：

- 读数据集和版本。
- 构造 prompt 或工具环境。
- 调用模型/API。
- 控制 temperature、top-p、seed、max tokens、thinking budget。
- 管理工具、权限、sandbox、timeout。
- 解析输出。
- 调用 scorer、oracle 或 judge。
- 保存 trace、日志、成本和错误。
- 生成报告。

面试表达：

Benchmark 是题，metric 是怎么算分，harness 是把题以固定协议喂给模型并记录结果的执行系统。对

agent 来说，harness 还包含工具、状态、权限、恢复和 trace，所以分数不能只归因于 base model。

Benchmark、Harness、Scaffold 的区别

概念	English	中文	关注点
Benchmark	Benchmark	基准	任务集合、标准答案、指标。
Harness	Harness	评测/执行框架	如何执行任务、调用模型、打分、记录。
Scaffold	Scaffold	外部脚手架	agent 的 planner、memory、tools、verifier、retry 逻辑。
Leaderboard	Leaderboard	排行榜	按某协议汇总比较分数。
Protocol	Protocol	协议	是否允许工具、预算、采样次数、prompt 格式。

一个模型的分数可以抽象成：

$$Score = f(M, H, T, P, B)$$

其中 M 是模型， H 是 harness， T 是任务集合， P 是 prompt/protocol， B 是预算。只说“某模型在某榜单多少分”往往信息不足。

LM Evaluation Harness

EleutherAI 的 LM Evaluation Harness 是经典 model eval harness。它把很多文本 benchmark 统一到同一套接口里。

核心组件：

- model adapter：Hugging Face、vLLM、SGLang、API model。
- task config：数据集、prompt、few-shot、metric。
- runner：批量执行。
- output parser：解析模型输出。
- metric：accuracy、exact match、F1、perplexity 等。

适合：

- 基础语言理解。
- few-shot / zero-shot benchmark。
- 开源模型横向比较。
- 回归测试某次训练或微调是否退化。

局限：

- 很多任务是静态文本，不覆盖工具使用。

- 对 reasoning model，要处理 hidden CoT / think_end_token。
- 对 agentic 系统，单轮 harness 不够。

OpenAI Evals 与自定义 Eval

OpenAI Evals 代表另一类思路：把 eval 当成产品和模型迭代的一部分。

典型结构：

1. 样本：输入、期望行为、评分标准。
2. Completion function：调用某个模型或系统。
3. Eval class：执行样本并收集输出。
4. Scorer：程序打分或 LLM-as-a-judge。
5. Report：聚合错误类型和样例。

面试重点：

生产 eval 不是只跑 MMLU，而是把真实用户失败样本、红队样本、权限边界样本、格式约束样本持续加入回归集。

Agent Harness

Agent harness 比普通 model harness 多很多东西。

模块	English	中文	作用
Context Manager	Context Manager	上下文管理器	决定给模型看哪些历史、文件和工具结果。
Tool Runtime	Tool Runtime	工具运行时	执行 shell、browser、database、editor。
State Store	State Store	状态存储	保存任务进度、文件 diff、memory。
Permission Gate	Permission Gate	权限门	控制写操作、网络、支付、删除等风险动作。
Recovery Policy	Recovery Policy	恢复策略	工具失败、超时、循环时如何处理。
Trace Logger	Trace Logger	轨迹日志	记录每步动作和观察。
Validator	Validator	验证器	检查最终 artifact、测试结果、业务状态。

Harness-Bench 这类研究强调：同一个 base model 在不同 harness 下可能表现差很多。因此报告 agent 能力时要说明：

- 模型版本。

- harness 版本。
- 工具集合。
- 上下文策略。
- 是否允许并行尝试。
- 时间和 token 预算。
- 是否有 reviewer/verifier。
- 最终如何判定成功。

Coding Agent Harness

Coding agent 的 harness 通常包含：

1. 仓库快照：固定 commit。
2. Issue 描述：用户需求或真实 bug。
3. Sandbox：隔离执行代码。
4. Tool set：搜索、编辑、测试、lint、类型检查。
5. Patch capture：记录最终 diff。
6. Hidden tests：避免只拟合公开测试。
7. Validator：运行测试或人工验证。
8. Trace：记录每次命令、文件读取和修改。

评测指标：

- pass@1。
- pass@k。
- resolved rate。
- regression rate。
- tool error rate。
- average cost。
- time-to-resolution。
- patch size。

面试表达：

SWE-bench 分数不只是模型能力，还取决于 scaffold、工具、测试预算、重试策略和 patch 验证方式。比较 coding agent 时必须看 protocol。

RAG Harness

RAG harness 需要同时固定 corpus、query、权限、gold evidence 和 judge。

最小结构：

- corpus version：文档库版本。
- index version：chunking、embedding、reranker 版本。
- query set：用户问题集合。

- gold evidence: 标准证据块。
- expected answer: 标准答案或评分 rubrics。
- permission scope: 每个 query 允许访问哪些文档。
- scorer: retrieval scorer、generation scorer、citation scorer。

RAG harness 输出不应该只有 answer accuracy, 还要有:

- Recall@k。
- MRR。
- citation precision。
- unsupported claim rate。
- refusal precision / recall。
- latency。
- cost。
- permission violation rate。

LLM-as-a-Judge 怎么用

LLM-as-a-Judge 适合开放式答案, 但不能裸用。

要注意:

- 先用人工样本校准 judge。
- prompt 里明确评分 rubrics。
- 使用 pairwise comparison 通常比绝对打分更稳定。
- 对 judge 本身做版本固定。
- 对安全和权限问题优先用规则或 oracle。
- 记录 judge rationale, 但不要把 rationale 当真值。

常见问题:

- position bias: 更偏好第一个答案。
- verbosity bias: 更偏好长答案。
- self-preference: 偏好同系列模型输出。
- rubric drift: 评分标准不稳定。
- evidence blindness: 没有认真核对引用。

污染、泄漏与可比性

2026 年 benchmark 最大的问题之一是可比性。

常见风险:

- test contamination: 测试集进入预训练或微调数据。
- prompt leakage: 评测 prompt 在网上公开并被训练。
- overfitting leaderboard: 专门优化公开榜。
- hidden scaffold: 不同模型使用了不同外部工具或预算。
- pass@k abuse: 用大量采样把分数堆高, 却不报告成本。

- benchmark saturation: 题目太简单, 区分不了前沿模型。

报告结果时至少说明:

1. 模型版本。
2. harness 版本。
3. benchmark 版本。
4. prompt 模板。
5. decoding 参数。
6. tool access。
7. thinking budget。
8. retries / pass@k。
9. 成本和延迟。
0. 是否做 decontamination。

Eval 研发闭环

一个成熟团队不会只在发布前跑一次评测, 而是建立闭环:

线上失败样本 -> 标注/归因 -> 加入 eval set -> 修模型/修系统 -> CI 回归 -> 灰度监控

评测集应该分层:

- smoke eval: 几十条, 提交前快速跑。
- regression eval: 几百到几千条, 覆盖核心场景。
- red-team eval: 安全、越权、prompt injection。
- canary eval: 新模型或新 prompt 灰度前跑。
- online eval: 真实流量抽检和人工反馈。

面试表达:

Eval 的价值不是得一个分数, 而是形成 failure taxonomy 和回归闭环。每次线上事故或人工纠错都应该沉淀成可复现 eval。

面试高频问题

1. Harness 和 benchmark 有什么区别?

Benchmark 是题和指标, harness 是执行系统。Harness 决定 prompt、工具、预算、解析、打分、日志和复现方式。

2. 为什么 agent 要报告 model-harness 组合?

因为 agent 能力强依赖工具、context management、retry、verifier、sandbox 和权限策略。同一模型换 harness, 成功率和失败模式都可能变。

3. 怎么设计一个 RAG eval harness?

固定 corpus/index/query/gold evidence/permission scope, 分别评估 retrieval、rerank、generation、citation、refusal、latency、cost 和越权率。

4. LLM-as-a-Judge 靠谱吗?

可以用于趋势和开放式评测, 但要用人工样本校准, 固定 judge 版本, 控制位置偏差和冗长偏差; 高风险结论要用规则、oracle 或人工复核。

5. 为什么排行榜分数不能直接横比?

因为 prompt、decoding、thinking budget、tool access、scaffold、pass@k 和测试污染都可能不同。面试里要先问 protocol, 再看分数。

本章 References

- [EleutherAI LM Evaluation Harness](#)
- [OpenAI Evals](#)
- [HELM: Holistic Evaluation of Language Models](#)
- [VHELM: A Holistic Evaluation of Vision Language Models](#)
- [Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows](#)
- [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)
- [TAU-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains](#)

LLM / VLM References

LLM 基础、Scaling 与架构

- Vaswani et al., 2017, [Attention Is All You Need](#)
- Brown et al., 2020, [Language Models are Few-Shot Learners](#)
- Kaplan et al., 2020, [Scaling Laws for Neural Language Models](#)
- Hoffmann et al., 2022, [Training Compute-Optimal Large Language Models](#)
- Dao et al., 2022, [FlashAttention](#)
- Meta AI, 2024, [The Llama 3 Herd of Models](#)
- Meta AI, 2025, [Llama 4 multimodal intelligence](#)
- Google, 2025, [Gemini 2.5 Technical Report](#)
- Qwen Team, 2024, [Qwen2.5 Technical Report](#)
- Qwen Team, 2025, [Qwen3 Technical Report](#)

Post-training、Alignment 与 Reasoning

- Ouyang et al., 2022, [Training language models to follow instructions with human feedback](#)
- Bai et al., 2022, [Constitutional AI: Harmlessness from AI Feedback](#)
- Lightman et al., 2023, [Let's Verify Step by Step](#)
- Rafailov et al., 2023, [Direct Preference Optimization](#)
- Shao et al., 2024, [DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models](#)
- Guan et al., 2024, [Deliberative Alignment: Reasoning Enables Safer Language Models](#)
- DeepSeek-AI, 2025, [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)
- OpenAI, 2025, [OpenAI o3 and o4-mini System Card](#)
- OpenAI, 2025, [Introducing OpenAI o3 and o4-mini](#)
- OpenAI, 2026, [OpenAI GPT-5 System Card](#)
- Gould et al., 2026, [Think Fast: Estimating No-CoT Task-Completion Time Horizons of Frontier AI Models](#)

PEFT 与高效微调

- Hu et al., 2021, [LoRA](#)
- Dettmers et al., 2023, [QLoRA](#)

Prompt、RAG、Agent 与 Eval

- Wei et al., 2022, [Chain-of-Thought Prompting](#)
- Wang et al., 2022, [Self-Consistency Improves Chain of Thought Reasoning](#)

- Lewis et al., 2020, [Retrieval-Augmented Generation](#)
- Guu et al., 2020, [REALM](#)
- Microsoft, [GraphRAG](#)
- VibrantLabs, [RAGAS](#)
- Singh et al., 2025, [Agentic Retrieval-Augmented Generation](#)
- Sharma, 2025, [Retrieval-Augmented Generation: A Comprehensive Survey](#)
- Schick et al., 2023, [Toolformer](#)
- Yao et al., 2022, [ReAct](#)
- Liu et al., 2023, [AgentBench](#)
- Jimenez et al., 2023, [SWE-bench](#)
- Yao et al., 2024, [TAU-bench](#)
- Wei et al., 2025, [BrowseComp](#)
- Kapoor et al., 2026, [Open-World Evaluations for Measuring Frontier AI Capabilities](#)
- Liang et al., 2022, [HELM](#)
- EleutherAI, [LM Evaluation Harness](#)
- OpenAI, [Evals](#)
- Lee et al., 2024, [VHELM](#)
- Yao et al., 2026, [Harness-Bench](#)
- Muennighoff et al., 2022, [MTEB](#)
- Hendrycks et al., 2020, [MMLU](#)

2025-2026 SOTA 技术报告和官方模型页

- OpenAI, 2025, [GPT-5 is here](#)
- OpenAI, 2026, [Introducing GPT-5.5](#)
- Anthropic, 2025, [Claude 4](#)
- Anthropic, 2026, [Claude Opus 4.7](#)
- Google, 2025, [Gemini 2.5 Technical Report](#)
- DeepSeek-AI, 2025, [DeepSeek-R1](#)
- Qwen Team, 2025, [Qwen3 Technical Report](#)
- Kimi Team, 2025, [Kimi K2: Open Agentic Intelligence](#)
- Kimi Team, 2026, [Kimi K2.5: Visual Agentic Intelligence](#)
- GLM Team, 2026, [GLM-5: from Vibe Coding to Agentic Engineering](#)
- Z.ai, 2026, [GLM-5.2 / GLM-5.1 / GLM-5 official repo](#)
- GLM Team, 2025, [GLM-4.5: Agentic, Reasoning, and Coding Foundation Models](#)
- GLM-V Team, 2025, [GLM-4.1V-Thinking and GLM-4.5V](#)

VLM / MLLM

- Dosovitskiy et al., 2020, [Vision Transformer](#)
- Radford et al., 2021, [CLIP](#)
- Alayrac et al., 2022, [Flamingo](#)
- Li et al., 2023, [BLIP-2](#)

- Liu et al., 2023, [LLaVA: Visual Instruction Tuning](#)
- Bai et al., 2025, [Qwen2.5-VL Technical Report](#)
- Bai et al., 2025, [Qwen2.5 Technical Report](#)
- OpenAI, 2023, [GPT-4 Technical Report](#)
- Liu et al., 2023, [MMBench](#)
- Yue et al., 2023, [MMMU](#)